

CSE 753 — Advanced Reinforcement Learning

Comprehensive Final Examination Study Notes

Lectures 06–09: Policy Gradients, Actor–Critic, Imitation Learning and Offline RL, Reward Learning and RLHF

Prepared as course study material

Contents

1	Part I — Policy Gradient Methods (Lecture 06)	3
1.1	The setting: why learn a policy directly at all?	3
1.2	Policy parameterisation and the softmax policy	3
1.3	Why stochastic policies: the state-aggregation argument	4
1.4	The RL objective and its gradient	4
1.5	REINFORCE (vanilla policy gradient)	5
1.6	Reward-to-go	6
1.7	Baselines	7
1.8	On-policy invalidity — the boundary you must hold	8
1.9	Importance sampling and the off-policy policy gradient	8
1.10	The KL constraint — the bridge to Lecture 07	9
2	Part II — Actor–Critic Methods (Lecture 07)	10
2.1	The three value objects	10
2.2	The pivot: policy gradient becomes actor–critic	10
2.3	The one-step TD error — the most-tested formula in the course	11
2.4	Generalised Advantage Estimation (GAE)	12
2.5	The surrogate objective	12
2.6	Proximal Policy Optimization (PPO)	13
2.7	The off-policy actor–critic ladder	14
2.8	Why fitting Q fixes what fitting V could not	15
2.9	Why $\bar{a}$ is re-sampled from π_θ rather than read from the buffer	15
3	Part III — Imitation Learning and Offline RL (Lecture 08)	16
3.1	Imitation learning: the setting	16
3.2	Behavioural cloning version 0: ℓ_2 regression, and the mean-seeking failure	16
3.3	Behavioural cloning version 1: expressive policies	17
3.4	Compounding error and distribution shift	19
3.5	Dagger (Dataset Aggregation)	20
3.6	Why offline RL?	20
3.7	Offline RL versus imitation learning: the stitching argument	21
3.8	Why naive off-policy Q-learning fails offline: the OOD chain	22
3.9	Filtered behavioural cloning	22
3.10	Weighted imitation learning and AWR	23
3.11	Advantage-Weighted Actor–Critic (AWAC)	24
3.12	Expectile regression	25
3.13	Implicit Q-Learning (IQL)	25
3.14	The four offline methods as one progression	26

4	Part IV — Reward Learning and RLHF (Lecture 09)	27
4.1	Where does the reward come from?	27
4.2	Goal classifiers	27
4.3	Learning rewards from human preferences	28
4.4	The three-stage RLHF pipeline	29
4.5	The RLHF objective and the KL penalty	29
4.6	Direct Preference Optimization (DPO)	30
4.7	RLAIF / Constitutional RL	31
4.8	Goal classifiers versus RLHF — the comparison table	32
5	Part V — The Threads Running Through All Four Lectures	33
5.1	Thread 1 — Every fix introduces a new failure	33
5.2	Thread 2 — Proxy exploitation is one idea appearing three times	33
5.3	Thread 3 — Bias versus variance in bootstrapping	34
5.4	Thread 4 — Everything offline is weighted behaviour cloning	34
6	Errors and Ambiguities in the Source Materials	35
6.1	(a) Scenario 1 Q3 contains a false premise — do not accept it	35
6.2	(b) Scenario 6 Q4: the expectile convention is reversed relative to the standard	35
6.3	(c) The “Diffusion Policy” slide describes flow matching	35
6.4	(d) Lecture 06 slides 11 and 13 share the same answer text	36
6.5	(e) Lecture 09 slide 8 has a typo in the preference inequality	36
6.6	(f) Scenario 5 Q2 notation	36
6.7	(g) A note on the practice paper’s Section 1 heading	36
7	Worked Solutions to the Practice Paper	37
7.1	Scenario 1 — The Warehouse Robot	37
7.2	Scenario 2 — The Traffic Signal Controller	37
7.3	Scenario 3 — The Medical Dosing Agent	38
7.4	Scenario 4 — The Robot Chef	38
7.5	Scenario 5 — The Surgical Robot	39
7.6	Scenario 6 — The Autonomous Submarine	39
7.7	Scenario 7 — The Logistics Drone	40
7.8	Scenario 8 — The Content Moderation AI	40
7.9	Scenario 9 — The Chatbot Alignment Team	41
7.10	Scenario 10 — The Robot Goal Classifier	42
8	Compressed Formula Sheet	44
9	Symbol Table	45
10	Final Revision Checklist	46

1 Part I — Policy Gradient Methods (Lecture 06)

1.1 The setting: why learn a policy directly at all?

What it is. Everything before this lecture was *generalised policy iteration*: maintain $Q(s, a)$, and derive the policy from it by acting greedily. Policy gradient methods throw the intermediate step away. You parameterise the policy itself as $\pi(a | s, \theta) = \Pr\{A_t = a | S_t = s, \theta_t = \theta\}$, with $\theta \in \mathbb{R}^d$, and improve θ by gradient ascent on performance.

Plain English. Instead of learning “how good is each action here” and then picking the best, you learn “what should I do here” directly, and nudge that behaviour up or down according to how well things turned out.

What problem it solves. Three things. (i) In continuous or very large action spaces, $\arg \max_a Q(s, a)$ is itself a hard optimisation problem at every step; a parameterised policy just outputs the action. (ii) The policy is often a simpler object than the value function — knowing *that* you should brake is easier than knowing the exact discounted return of braking. (iii) It gives you *stochastic* policies natively, which value-greedy methods cannot represent.

What it cannot solve. It gives no value estimate, so it has no mechanism for credit assignment beyond “the whole trajectory turned out well”. That gap is precisely what Lecture 07 fills.

What problems it gave rise to. High variance, on-policy-only data, and sample inefficiency — the three themes of the rest of the lecture.

1.2 Policy parameterisation and the softmax policy

What it is. Any parameterisation must satisfy $\pi(a | s, \theta) \geq 0$ for all a, s and $\sum_{a \in \mathcal{A}} \pi(a | s, \theta) = 1$ for all s . For discrete actions the standard construction is the softmax over *action preferences* $h(s, a, \theta)$:

$$\pi(a | s, \theta) = \frac{e^{h(s, a, \theta)}}{\sum_{b \in \mathcal{A}} e^{h(s, b, \theta)}}.$$

Plain English. The network outputs one unnormalised score per action; the exponential makes those scores positive and the denominator makes them sum to one. The scores are called preferences because their *absolute* value is meaningless — only differences between them matter.

The key behavioural fact. As the preference $h(s, a, \theta)$ of one action grows large relative to the others, the softmax tends towards *greedy*: it puts almost all probability on that action. This is the link between parameter magnitude and exploration. A policy that has been trained for a long time has large preferences and therefore explores little, without anyone having explicitly scheduled an exploration parameter.

Advantages. Smooth in θ , so gradients exist everywhere; naturally normalised; can approach determinism arbitrarily closely without ever becoming discontinuous.

Disadvantages. It can *only* approach determinism asymptotically, so it can be slow to commit; and because only differences of preferences matter, the parameterisation is over-specified (adding a constant to every h changes nothing), which can make optimisation ill-conditioned.

What if it had been done differently? If you parameterised the probabilities directly and projected onto the simplex, you would have to handle the constraint explicitly at every step and you would lose differentiability at the simplex boundary. The exponential is there to turn a constrained problem into an unconstrained one.

Exam hook. “As the action preference $h(s, a, \theta)$ grows, how does the policy behave?” Answer: it tends towards greedy action selection, which also means exploration decays as parameters grow.

1.3 Why stochastic policies: the state-aggregation argument

What it is. A claimed advantage of policy parameterisation is that it can represent stochastic policies, and under function approximation the best stochastic policy can be *strictly better* than the best deterministic one.

The lecture’s example. A short corridor of three states leading to a goal G , reward -1 per step, where in the middle state the effects of the two actions are *reversed*. Crucially, all three states are *aggregated* — the function approximator cannot tell them apart, so it must emit the same action distribution in all three.

Plain English. If you cannot distinguish two states, and the right action differs between them, then *any* deterministic rule is wrong in at least one of them, and being wrong there can trap you (you loop forever). A coin-flipping policy is wrong sometimes in both, but never traps you. Mixing is strictly better than committing when you are blind.

The number to quote. The best stochastic policy under aggregation puts roughly 59% of its mass on “right” and 41% on “left”, achieving $V_{\pi_\theta}(s) = -11.6$. Both deterministic policies do far worse. (I verified this by solving the linear system for the aggregated chain: the optimum sits at $p \approx 0.59$ with $V \approx -11.66$.)

What problem it solves. Partial observability and function approximation error. Randomisation is a hedge against not knowing which state you are in.

What it cannot solve. It does not recover the information you lost. The stochastic optimum is still much worse than the true optimum with full state information. Randomising is damage limitation, not a fix.

When to use it. Whenever the representation cannot distinguish states that require different actions — partial observability, aggressive state aggregation, adversarial settings where predictability is exploitable.

When not to. In a fully observed, tabular MDP the optimal policy is deterministic; stochasticity there is pure loss.

Exam hook. A one-line justification part. Say: “under function approximation with state aggregation, the best deterministic policy can be strictly worse than the best stochastic one — the corridor example gives $V_{\pi_\theta} = -11.6$ for the 41%/59% mixture.”

1.4 The RL objective and its gradient

The objective.

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{\tau \sim p_{\theta}(\tau)}[r(\tau)], \quad J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(s_t, a_t) \right] \approx \frac{1}{N} \sum_i \sum_t r(s_{i,t}, a_{i,t}),$$

where the approximation is a Monte Carlo average over N trajectories sampled *from* π_θ . That last clause is the whole on-policy story in miniature.

The derivation — reproduce this chain.

$$J(\theta) = \int p_\theta(\tau) r(\tau) d\tau \quad (1)$$

$$\nabla_\theta J(\theta) = \int \nabla_\theta p_\theta(\tau) r(\tau) d\tau \quad (\text{reward does not depend on } \theta) \quad (2)$$

$$= \int p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) r(\tau) d\tau \quad (\text{log-derivative trick}) \quad (3)$$

$$= \mathbb{E}_{\tau \sim p_\theta} [\nabla_\theta \log p_\theta(\tau) r(\tau)]. \quad (4)$$

The log-derivative trick is the identity $p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) = p_\theta(\tau) \frac{\nabla_\theta p_\theta(\tau)}{p_\theta(\tau)} = \nabla_\theta p_\theta(\tau)$. Its purpose is to turn $\int \nabla_\theta p_\theta(\tau) (\cdot) d\tau$, which is not an expectation and cannot be sampled, back into an expectation under p_θ , which *can* be sampled.

Why the dynamics vanish — say this in one sentence. Expand

$$\log p_\theta(\tau) = \log p(s_1) + \sum_{t=1}^T \log \pi_\theta(a_t | s_t) + \sum_{t=1}^T \log p(s_{t+1} | s_t, a_t).$$

The initial-state term and the transition terms do not depend on θ , so they die under ∇_θ . **This is why policy gradient is model-free:** you never need to know or estimate $p(s_{t+1} | s_t, a_t)$.

The final estimator.

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_\theta \log \pi_\theta(a_t^i | s_t^i) \right) \left(\sum_{t=1}^T r(s_t^i, a_t^i) \right).$$

How to read the estimator (this single sentence answers most of Section 1). It is a **weighted maximum-likelihood update, not a descent on a loss**. The vector $\nabla_\theta \log \pi_\theta(a | s)$ is the direction in parameter space that makes the action you actually took *more likely*; the return is a scalar deciding how far to move in that direction and with what sign. Positive return $\Rightarrow$ do more of that. Negative return $\Rightarrow$ do less.

Properties. The estimator is **unbiased** but **high-variance**. Every later technique in this course trades a little bias for a large reduction in variance, or spends computation to reduce variance without adding bias.

1.5 REINFORCE (vanilla policy gradient)

The algorithm.

1. Sample $\{\tau^i\}$ from $\pi_\theta(a_t | s_t)$.
2. $\nabla_\theta J(\theta) \approx \sum_i \left(\sum_{t=1}^T \nabla_\theta \log \pi_\theta(a_t^i | s_t^i) \right) \left(\sum_{t=1}^T r(s_t^i, a_t^i) \right)$.
3. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$.
4. Repeat until convergence.

What problem it solves. It gives an unbiased, model-free way to improve a policy in an arbitrary MDP, including continuous actions, with no value function and no knowledge of dynamics.

What it cannot solve.

- **Credit assignment within a trajectory.** Every action in a trajectory gets the *same* scalar weight — the total return. A good action in a bad trajectory is punished; a bad action in a good trajectory is reinforced.
- **Variance.** The weight is a sum of many random rewards, so its spread is enormous, and the gradient estimate is dominated by noise.
- **Reward-scale sensitivity.** Add +1000 to every reward and the optimal policy is unchanged, but every weight becomes hugely positive and the algorithm reinforces everything, including failures.
- **Sample reuse.** It is on-policy; a single batch buys a single gradient step.

Exam hook — the three quiz slides, which are three distinct failures.

Symptom	Cause	Fix on the slides
Whole trajectory uniformly reinforced or punished; a step forward gets pushed down because the robot later fell backwards	No credit assignment across t ; every action weighted by the total return	Reward-to-go
All four trajectories succeed, so all weights are positive and behaviour that is merely less bad is still reinforced	Returns are all positive; the estimator is noisy and sensitive to reward scale	Baseline
Jacket folding with $r \in \{0, 0.5, 1\}$: gradient weight is essentially constant across trajectories	Sparse reward — almost every trajectory receives 0	Baseline helps, but the real fix is a learned critic (Lecture 07)

Note on the second row. The lecture slide for the all-successes case reuses the answer text of the first slide. Do not copy that first sentence; the actual point of that slide is the *second* sentence: policy gradient is noisy, high-variance and sensitive to reward scale.

1.6 Reward-to-go

What it is. Replace the trajectory return in the weight by the sum of rewards *from the current timestep onwards*:

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_i \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \left(\sum_{t'=t}^T r(s_{t'}^i, a_{t'}^i) \right), \quad \hat{Q}_t^i = \sum_{t'=t}^T r(s_{t'}^i, a_{t'}^i).$$

Note the limit: the sum starts at $t' = t$, **inclusive**.

Plain English. Judge an action by what happened after it, not by what happened before it. Anything that occurred earlier in the trajectory is not that action's fault or credit.

The justification, in one line. *Policy behaviour at time t cannot affect rewards at times $t' < t$ — causality.* Dropping those terms therefore removes noise without introducing bias.

What problem it solves. Partial credit assignment across time, and a genuine variance reduction (fewer random terms in each weight).

What it cannot solve. It still cannot separate a *bad action* from a *good outcome that happened to follow it*. If an action moves you away from the goal but a large reward arrives two steps later, reward-to-go credits the bad action with that reward. This is exactly the trap in Scenario 1 Q3 of the practice paper.

What problem it gave rise to. Nothing new — it is a strict improvement. But by making the weights *differ* across timesteps it exposes the next issue: the weights are still all positive when all rewards are positive.

Counterfactual. If you summed from $t + 1$ instead of t , you would be judging the action by everything except its own immediate consequence — throwing away the most informative single term. If you summed from $t' = 1$ you would be back to REINFORCE. The inclusive lower limit is load-bearing; the examiner checks it.

Worked pattern (practice paper). Rewards $\tau_1 = (+1, +10)$, $\tau_2 = (-1, +10)$, $\tau_3 = (+1, -1)$, $\tau_4 = (-1, -1)$.

$$\hat{Q}^1 = (11, 10), \quad \hat{Q}^2 = (9, 10), \quad \hat{Q}^3 = (0, -1), \quad \hat{Q}^4 = (-2, -1).$$

Practise building this table quickly: the $t = 2$ entry is just r_2 ; the $t = 1$ entry is $r_1 + r_2$.

1.7 Baselines

What it is. Subtract a constant from the weight:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}} [\nabla_{\theta} \log p_{\theta}(\tau) (r(\tau) - b)], \quad b = \frac{1}{N} \sum_{i=1}^N r(\tau_i).$$

Plain English. “Was this better or worse than average?” rather than “was this good?”. Only above-average behaviour gets reinforced; below-average behaviour gets actively pushed down, even when its raw return was positive.

The unbiasedness proof — memorise the four-step chain.

$$\mathbb{E}[\nabla_{\theta} \log p_{\theta}(\tau) b] = \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) b \, d\tau = \int \nabla_{\theta} p_{\theta}(\tau) b \, d\tau = b \nabla_{\theta} \int p_{\theta}(\tau) \, d\tau = b \nabla_{\theta} 1 = 0.$$

The load-bearing step is the last one: the constant survives the expectation only because the density integrates to one, so its gradient is the gradient of a constant. **Subtracting a constant baseline changes the variance but not the expectation.**

What problem it solves. (i) Removes the “everything is positive, so reinforce everything” failure. (ii) Reduces variance. (iii) Removes sensitivity to an additive shift of the reward function.

What it cannot solve.

- It **reduces** variance; it does not remove it. Say “reduce”, not “eliminate”.
- A *constant* baseline is state-independent. It cannot answer “was this action better than what my policy usually does *in this state*?”, which is the question that actually matters for credit assignment. That gap is what the advantage function $A^{\pi}(s, a)$ in Lecture 07 exists to close.
- Under sparse rewards the baselined weight is near-constant across almost all trajectories, so it barely helps. This motivates a learned critic.

Counterfactual. If b depended on the *action*, the proof would fail: $b(a)$ could not be pulled outside the integral over τ and the estimator would become biased. This is precisely why the baseline may depend on the state (giving $V^\pi(s)$ and hence the advantage) but never on the action. That is a strong E4 answer if asked why V and not Q is used as a baseline.

Sign reading (E2 — this is most of Section 1).

$$\hat{Q}_t^i - b > 0 \Rightarrow \text{gradient increases } \log \pi_\theta(a_t^i | s_t^i); \quad \hat{Q}_t^i - b < 0 \Rightarrow \text{it decreases it.}$$

Worked pattern. Continuing the warehouse table: the eight reward-to-go samples are $\{11, 10, 9, 10, 0, -1, -2, -1\}$, so $b = \frac{36}{8} = 4.5$. Then, for example, τ_3 at $t = 1$ has $0 - 4.5 = -4.5 < 0$ (log-probability *decreased*) and τ_4 at $t = 2$ has $-1 - 4.5 = -5.5 < 0$ (also decreased).

1.8 On-policy invalidity — the boundary you must hold

What it is. $J(\theta)$ is an expectation over $\tau \sim p_\theta$. The Monte Carlo estimate is only valid for the θ that generated the samples. The moment you take one gradient step, θ has moved, and the stored trajectories are draws from the *wrong* distribution.

Say it precisely. Reusing a batch is not merely *inefficient*; the estimator becomes **biased** — it is an estimator of the gradient of the wrong objective. This distinction is what a question about “reuse the four trajectories for five more gradient steps” is testing.

Consequence. One batch of environment interaction buys exactly one gradient step. For a real robot or a hospital, this is prohibitively expensive.

1.9 Importance sampling and the off-policy policy gradient

What it is. Reweight each old sample by how much more or less likely it is under the new policy. At the trajectory level:

$$\nabla_{\theta'} J(\theta') = \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\prod_{t=1}^T \frac{\pi_{\theta'}(a_t | s_t)}{\pi_\theta(a_t | s_t)} \left(\sum_{t'=1}^T \nabla_{\theta'} \log \pi_{\theta'}(a_{t'} | s_{t'}) \right) \left(\sum_{t'=t}^T r(s_{t'}, a_{t'}) - b \right) \right].$$

Plain English. If the new policy would have chosen this action twice as often as the old one, count this sample twice. That converts data from the old policy into a valid (if noisier) estimate for the new one.

The failure — this is the standard E5 answer. The trajectory ratio is a **product of T terms**. A product of many numbers each slightly above or below one either explodes or collapses towards zero, and its *variance grows exponentially in the horizon*. State it as: *for long trajectories, or for a policy that has moved far from π_θ , the product of T ratios has variance growing exponentially with T , so the estimate becomes useless.*

The per-timestep repair (the final form on the slides).

$$\nabla_{\theta'} J(\theta') \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \frac{\pi_{\theta'}(a_{i,t} | s_{i,t})}{\pi_\theta(a_{i,t} | s_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(a_{i,t} | s_{i,t}) \left(\sum_{t'=t}^T r(s_{i,t'}, a_{i,t'}) - b \right).$$

The slides pass through an intermediate form with a *state-action marginal* ratio $\frac{\pi_{\theta'}(s_{i,t}, a_{i,t})}{\pi_\theta(s_{i,t}, a_{i,t})}$, note that it is “much less likely to explode or vanish, but hard to measure” — you cannot compute a state marginal — and then drop the state part to reach the action-only form above. That dropped factor is an *approximation*: the final estimator is no longer exactly unbiased.

What it cannot solve. Per-timestep ratios tame the explosion but do not make the method safe. It is trustworthy only while $\pi_{\theta'} \approx \pi_{\theta}$. Reweighting cannot manufacture information about states the old policy never visited: if $\pi_{\theta'}$ would go somewhere π_{θ} never went, there are no samples there to reweight, and no amount of arithmetic will produce them.

What problem it gave rise to. A new incentive problem. Now that multiple gradient steps on one batch are permitted, the policy is *rewarded* for drifting away from the data-collecting policy, because that is where the surrogate objective looks best. It will overfit the batch.

When to use it. When environment interaction is expensive and the policy moves slowly — short horizons, small learning rates, few steps per batch.

When not to. Long horizons; large policy updates; replay buffers containing data from arbitrarily old policies (there the weights are hopeless).

1.10 The KL constraint — the bridge to Lecture 07

What it is. If the ratio only misbehaves when the policies diverge, then constrain the divergence:

$$\mathbb{E}_{s \sim \pi_{\theta}} [D_{\text{KL}}(\pi_{\theta'}(\cdot | s) \| \pi_{\theta}(\cdot | s))] \leq \delta.$$

Plain English. Take as large a step as you like in parameter space, as long as the resulting *behaviour* has not changed by more than δ . The constraint is on the distribution over actions, not on $\|\theta' - \theta\|$ — because two parameter vectors can be close while the policies they induce are completely different.

The arc to present as one argument, not four facts. Reuse data $\rightarrow$ importance ratios explode $\rightarrow$ constrain the policy so they cannot $\rightarrow$ enforcing a hard KL constraint is awkward in practice $\rightarrow$ PPO implements a soft version by clipping the ratios instead.

Behaviour at the limits (a direct exam question).

- $\delta \rightarrow 0$: $\pi_{\theta'}$ is pinned to π_{θ} . Updates are effectively frozen. Maximally safe, zero progress.
- $\delta \rightarrow \infty$: the constraint is vacuous. You are back to unconstrained off-policy updates: the policy overfits the batch, the importance weights become unreliable, and the collected data no longer reflects the states the policy will actually visit.

The name to give. The practical algorithm that enforces a soft version of this constraint by *clipping importance weights* rather than constraining the KL is **PPO**. If a question describes a trust region and asks for “the practical algorithm”, it wants that word.

2 Part II — Actor–Critic Methods (Lecture 07)

2.1 The three value objects

Definitions.

- $V^\pi(s)$: expected future reward starting at s and following π .
- $Q^\pi(s, a)$: expected future reward starting at s , taking a *once*, then following π .
- $A^\pi(s, a)$: how much better taking a is than following π at s .

The two relations you must have automatic:

$$V^\pi(s) = \mathbb{E}_{a \sim \pi(\cdot|s)}[Q^\pi(s, a)], \quad A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s).$$

Plain English. Q evaluates a specific action; V averages Q over what the policy would actually do; A is the difference, i.e. “how much does deviating to a gain me relative to business as usual”. Because V is the π -average of Q , the advantage of the current policy averages to zero: $\mathbb{E}_{a \sim \pi}[A^\pi(s, a)] = 0$. That is a useful sanity check and occasionally an exam answer in its own right.

The lecture’s worked example (be able to do this in your head). Three actions — practise drums (a_1), study at a desk (a_2), lie on a deck chair (a_3) — reward 1 if you can play the drums in a month, 0 otherwise, and the current policy is $\pi(a_1 | s) = 1$ for all s . Then $V^\pi(s) = 1$ (the policy always practises, so it always succeeds); $Q^\pi(s, a_1) = 1$, $Q^\pi(s, a_2) = Q^\pi(s, a_3) = 0$; and therefore $A^\pi(s, a_1) = 0$, $A^\pi(s, a_2) = A^\pi(s, a_3) = -1$. Note the shape of the answer: the advantage of the action the policy already takes is zero. An advantage of zero does not mean the action is bad — it means it is exactly what the policy already does.

2.2 The pivot: policy gradient becomes actor–critic

The single insight of this lecture. Look again at the improved policy gradient:

$$\nabla_\theta J(\theta) \approx \sum_i \sum_t \nabla_\theta \log \pi_\theta(a_t^i | s_t^i) \underbrace{\left(\sum_{t'=t}^T r(s_{t'}, a_{t'}^i) - b \right)}_{\text{reward-to-go}}.$$

Reward-to-go is a **single-sample estimate of $Q^\pi(s_t, a_t)$** , since $\sum_{t'=t}^T \mathbb{E}_{\pi_\theta}[r(s_{t'}, a_{t'}) | s_t, a_t] = Q^\pi(s_t, a_t)$. The average-return baseline is a **crude estimate of $V^\pi(s_t)$** . Substituting learned functions for both:

$$\nabla_\theta J(\theta) \approx \sum_i \sum_t \nabla_\theta \log \pi_\theta(a_t^i | s_t^i) A^\pi(s_{i,t}, a_{i,t}).$$

Say it as a substitution, not as a new algorithm. Actor–critic is not a different family from policy gradient. It is policy gradient with the noisy one-sample Q replaced by a learned estimate and the crude constant baseline replaced by a learned V . The *actor* is π_θ ; the *critic* is $\hat{V}_\phi$ (or $\hat{Q}_\phi$).

What problem it solves.

- **Variance:** one high-variance sampled return is replaced by a low-variance function approximation that has pooled information across many trajectories.
- **Credit assignment:** the baseline is now *state-dependent*, so the question becomes “was this action better than what I usually do *here*” rather than “better than average overall”. This is exactly the limitation of the constant baseline in Lecture 06.
- **Sparse rewards:** the critic propagates value backwards from the rare rewarding states, so intermediate states acquire non-trivial values and the gradient weight stops being constant across trajectories.
- **Data efficiency:** the critic can be trained on all transitions, including ones that received zero reward.

What it cannot solve, and what it gave rise to. The estimator is now **biased**, because $\hat{V}_\phi$ is wrong early in training and never exactly right later. Worse, the bias is not benign noise: errors in $\hat{V}$ feed into the targets used to train $\hat{V}$, so they can compound. Every subsequent design in this course — GAE, target networks, expectile regression — is management of that bias.

2.3 The one-step TD error — the most-tested formula in the course

Derivation in three lines.

$$Q^\pi(s_t, a_t) = r(s_t, a_t) + \mathbb{E}_{s_{t+1} \sim p(\cdot | s_t, a_t)} [V^\pi(s_{t+1})] \quad (5)$$

$$\approx r(s_t, a_t) + V^\pi(s_{t+1}) \quad (\text{use the single sampled } s_{t+1}) \quad (6)$$

$$\Rightarrow A^\pi(s_t, a_t) \approx r(s_t, a_t) + \gamma V^\pi(s_{t+1}) - V^\pi(s_t). \quad (7)$$

The formula.

$\delta_t = r_t + \gamma \hat{V}(s_{t+1}) - \hat{V}(s_t), \quad \hat{A}_1^\pi(s_t, a_t) = \delta_t.$

Two facts that must be automatic.

- (i) **The one-step advantage is the TD error.** If a question asks for the “one-step advantage” after you have computed δ_t , the answer is δ_t . Do not compute anything extra.
- (ii) δ_t is a scalar; $\nabla_\theta \log \pi_\theta$ is a vector. A question that hands you a three-component gradient and a TD error is checking that you know which is which.

Plain English. δ_t is surprise. You expected $\hat{V}(s_t)$; what you actually got was one real reward plus your (discounted) estimate of where you ended up. The difference is how much better or worse things went than the critic predicted. Positive surprise $\Rightarrow$ raise the critic’s estimate *and* make the action more likely.

One quantity, three jobs. δ_t is simultaneously

- the critic’s training signal: $\hat{V}'(s_t) = \hat{V}(s_t) + \alpha_{\text{critic}} \delta_t$;
- the one-step advantage estimate;
- the actor’s update weight: $\theta' = \theta + \alpha_{\text{actor}} \delta_t \nabla_\theta \log \pi_\theta(a_t | s_t)$.

Exam questions exploit this: they make you compute δ_t once and then use it twice.

Bias–variance position. The one-step advantage is **maximally biased and minimally variant**: it leans entirely on $\hat{V}$, which is wrong, but it contains only one sampled reward. That is the entry point to GAE.

The full actor–critic algorithm.

1. Sample a batch $\{(s_{1,i}, a_{1,i}, \dots, s_{T,i}, a_{T,i})\}$ from π_θ .
2. Fit $\hat{V}_\phi^{\pi_\theta}$ to the summed rewards in the data.
3. Evaluate $\hat{A}^{\pi_\theta}(s_{i,t}, a_{i,t}) = r(s_{i,t}, a_{i,t}) + \hat{V}_\phi^{\pi_\theta}(s_{i,t+1}) - \hat{V}_\phi^{\pi_\theta}(s_{i,t})$.
4. Evaluate $\nabla_\theta J(\theta) \approx \sum_i \sum_t \nabla_\theta \log \pi_\theta(a_t^i | s_t^i) \hat{A}^{\pi_\theta}(s_{i,t}, a_{i,t})$.
5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$.

Worked pattern. $r_t = +6$, $\hat{V}(s_t) = 20$, $\hat{V}(s_{t+1}) = 24$, $\gamma = 0.95$, $\alpha_{\text{critic}} = 0.1$, $\alpha_{\text{actor}} = 0.05$, $\nabla_{\theta} \log \pi_{\theta} = [0.6, -0.4, 0.2]$, $\theta = [3.0, 1.5, 2.0]$:

$$\begin{aligned}\delta_t &= 6 + 0.95(24) - 20 = 6 + 22.8 - 20 = 8.8, \\ \hat{V}'(s_t) &= 20 + 0.1(8.8) = 20.88, \\ \theta' &= [3.0, 1.5, 2.0] + 0.05(8.8)[0.6, -0.4, 0.2] \\ &= [3.0, 1.5, 2.0] + 0.44[0.6, -0.4, 0.2] = [3.264, 1.324, 2.088].\end{aligned}$$

2.4 Generalised Advantage Estimation (GAE)

What it is. The n -step advantage and its exponentially weighted average:

$$\hat{A}_n^{\pi}(s_t, a_t) = \sum_{t'=t}^{t+n} \gamma^{t'-t} r(s_{t'}, a_{t'}) - \hat{V}_{\phi}^{\pi}(s_t) + \gamma^n \hat{V}_{\phi}^{\pi}(s_{t+n}), \quad \hat{A}_{\text{GAE}}^{\pi}(s_t, a_t) = \sum_{n=1}^{\infty} w_n \hat{A}_n^{\pi}(s_t, a_t),$$

with $w_n \propto \lambda^{n-1}$ for $n = 1, \dots, N$ where $N \ll T$.

Plain English. Instead of trusting the critic after one step (low variance, high bias) or waiting for the whole episode (high variance, low bias), take a weighted blend of “trust the data for n steps, then hand over to the critic”. λ decides how quickly the blend hands over.

GAE is a dial, not an algorithm. Hold it that way:

- Small n or small λ : more bootstrapping. **Lower variance, higher bias.** The slide phrases it as “cut earlier for less variance”.
- Large n or $\lambda \rightarrow 1$: approaches Monte Carlo. **Lower bias, higher variance.**
- $n = 1$ recovers δ_t exactly.

Why the exponential weighting specifically? Because it makes an infinite sum computable and collapses to a single recursive pass. Any weighting would trade bias for variance; the geometric one is the choice that costs nothing to evaluate.

What problem it solves. It removes the forced choice between the two extremes of the bias-variance spectrum and turns it into a tunable hyperparameter.

What it cannot solve. It cannot make the critic correct. If $\hat{V}_{\phi}$ is systematically wrong, every n -step estimate inherits that error; blending biased estimates gives a biased blend. It also adds a hyperparameter that must be tuned per problem.

Counterfactual. If you fixed $\lambda = 0$ you would have plain one-step actor-critic: stable but potentially very biased in long-horizon tasks. If you fixed $\lambda = 1$ you would be back at REINFORCE-with-baseline: unbiased but too noisy to train large policies.

Exam hook. “You increase λ — what happens to bias and variance?” Bias down, variance up, moving towards Monte Carlo. And: a question asking for a “generalised advantage” with one step of data is answered with the TD error.

2.5 The surrogate objective

Where it comes from. Apply the identity $p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) = \nabla_{\theta} p_{\theta}(\tau)$ in reverse to the importance-weighted gradient

$$\nabla_{\theta'} J(\theta') \approx \sum_i \sum_t \frac{\pi_{\theta'}(a_{i,t} \mid s_{i,t})}{\pi_{\theta}(a_{i,t} \mid s_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(a_t^i \mid s_t^i) \hat{A}^{\pi_{\theta}}(s_{t,i}, a_{t,i})$$

to recover the *objective* whose gradient it is:

$$\tilde{J}(\theta') \approx \sum_{t,i} \frac{\pi_{\theta'}(a_{i,t} | s_{i,t})}{\pi_{\theta}(a_{i,t} | s_{i,t})} \hat{A}^{\pi_{\theta}}(s_{t,i}, a_{t,i}).$$

Plain English. Now you have something you can hand to an optimiser and run many gradient steps on, rather than a single gradient expression. That is the practical reason the surrogate exists.

What it gave rise to. Exactly the problem the extra machinery is built to contain: the objective *rewards* increasing the ratio wherever $\hat{A} > 0$, without limit. The policy is therefore incentivised to differ drastically from π_{θ} — and once it does, the advantages $\hat{A}^{\pi_{\theta}}$ it is being scored against are no longer valid.

2.6 Proximal Policy Optimization (PPO)

What it is. Off-policy actor-critic with three tricks bolted onto the surrogate objective. Writing $r_t(\theta') = \frac{\pi_{\theta'}(a_{t,i} | s_{t,i})}{\pi_{\theta}(a_{t,i} | s_{t,i})}$:

- **Trick 1 — clip the importance weights:** $\tilde{J}(\theta') \approx \sum_{t,i} \text{clip}(r_t(\theta'), 1 - \epsilon, 1 + \epsilon) \hat{A}$.
- **Trick 2 — take the minimum with the original objective:**

$$\tilde{J}(\theta') \approx \sum_{t,i} \min(r_t(\theta') \hat{A}, \text{clip}(r_t(\theta'), 1 - \epsilon, 1 + \epsilon) \hat{A}).$$

- **Trick 3 — GAE for $\hat{A}$.**

Plain English. “Improve the policy, but stop paying it for improvements that come from moving further than ϵ away from where the data came from.”

The distinction that E4 questions target: clipping does not constrain the policy. It removes the incentive to move far. Nothing prevents θ' from drifting; the objective simply stops improving once r_t leaves $[1 - \epsilon, 1 + \epsilon]$, so the gradient through that sample becomes zero and the optimiser has no reason to push further. Contrast with a KL constraint, which is a hard feasibility restriction. Say “removes incentives” rather than “enforces a constraint”.

Why the minimum exists (do not skip this). Clipping alone is one-sided. Consider $\hat{A} < 0$ with a ratio that has collapsed well below $1 - \epsilon$ — the policy has moved a long way in a harmful direction. Clipping the ratio up to $1 - \epsilon$ would make the objective look *better* than it is, hiding the damage. Taking the minimum makes the objective a **pessimistic lower bound**: the policy is never rewarded for having moved far in the harmful direction. The clip alone handles the $\hat{A} > 0$ side; the min is what handles the $\hat{A} < 0$ side.

Worked pattern (the compute-both-terms question). $r_1 = 1.5$, $\hat{A}_1 = +8$, $\epsilon = 0.2$:

$$\begin{aligned} \text{unclipped} &= r_1 \hat{A}_1 = 1.5 \times 8 = 12.0, \\ \text{clipped} &= \text{clip}(1.5, 0.8, 1.2) \times 8 = 1.2 \times 8 = 9.6, \\ \text{objective} &= \min(12.0, 9.6) = 9.6 \quad \Rightarrow \quad \text{the clipped term is used.} \end{aligned}$$

The *finding* to state (E2): the new policy has increased this action’s probability by more than ϵ allows, so the objective refuses to reward it any further — the update through this sample is throttled.

What problem PPO solves. It makes multiple gradient steps per batch practical and stable, without solving a constrained optimisation problem at every update (which is what TRPO does and why it is expensive).

What it cannot solve — the highest-value single fact in this part. PPO is a *proximal* method, not an *off-policy* method. Its surrogate is a valid local approximation to $J(\theta')$ only while $\pi_{\theta'}$ is close to the π_{θ} that generated the data. Given replay-buffer data from arbitrarily old policies, the ratios sit far outside $[1 - \epsilon, 1 + \epsilon]$, and clipping then *zeroes the gradient* rather than correcting the mismatch — you get no learning signal at all, not a corrected one. Importance sampling does not rescue this, because those weights have unusable variance. This is the answer to “why is PPO fundamentally unsuitable for a replay buffer”, and it should be one or two sentences.

What problems PPO gave rise to. Sensitivity to ϵ ; the possibility of premature convergence when clipping shuts off learning too aggressively; and a false sense that “PPO is off-policy”, which the exam is explicitly probing.

When to use it. Online RL with a simulator, where you can collect fresh batches and want several gradient steps out of each. **When not to.** Offline RL, replay buffers, or any setting where the behaviour policy is far from the current policy.

2.7 The off-policy actor–critic ladder

Learn this as a ladder, because “what breaks at each rung” is what gets asked.

Rung	What it does	What breaks
Fully on-policy	One batch, one gradient step	Sample-inefficient; every step costs fresh interaction
Multiple gradient steps	Importance-weighted reuse of one batch	The policy is incentivised to differ significantly from π_{θ} and can easily overfit the batch
Replay buffer, fit $\hat{V}$	Minibatch from all past data	(a) $\hat{V}$ is fit to “ <i>some kind of average policy</i> ”, so the target is wrong; (b) π_{θ} would not have taken a_i at s_i — wrong likelihood term
Replay buffer, fit $\hat{Q}$, resample $\bar{a}' \sim \pi_{\theta}$	Targets $y_i = r_i + \gamma \hat{Q}(s'_i, \bar{a}'_i)$; actor uses $\nabla_{\theta} \log \pi_{\theta}(\bar{a}_i s_i) \hat{Q}(s_i, \bar{a}_i)$	Works. Costs variance (no baseline), acceptable because there is now much more data. But it queries $\hat{Q}$ at actions the data may not contain — the seed of Lecture 08’s central problem

The two problems with a naive replay buffer (memorise both).

- (i) **The value target is wrong.** $V^{\pi}(s)$ is only defined *relative to a policy*. The buffer contains data from many past policies, so fitting $\hat{V}$ on all of it estimates the value of “some kind of average” of everything you have ever done — not of π_{θ} .
- (ii) **The likelihood term is wrong.** The actor update contains $\nabla_{\theta} \log \pi_{\theta}(a_i | s_i)$ where a_i came from an old policy. The current policy would probably not have taken a_i at s_i , so you are pushing probability towards actions the current policy has no reason to consider.

2.8 Why fitting Q fixes what fitting V could not

The design justification (a strong E4 answer). $V^\pi(s)$ is intrinsically policy-dependent — there is no way to read a V target off a transition without knowing whose policy generated the continuation. But $Q(s, a)$ *conditions on the action explicitly*, so a transition (s, a, r, s') from *any* policy is a valid training point for it:

$$Q^{\pi_\theta}(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim p(\cdot | s, a), \bar{a}' \sim \pi_\theta(\cdot | s')} [Q^{\pi_\theta}(s', \bar{a}')].$$

All of the policy-dependence has been pushed into one place — the choice of $\bar{a}'$ inside the target — and that is a place you *control*, by sampling $\bar{a}' \sim \pi_\theta(\cdot | s')$ yourself rather than reading it from the buffer.

The TD target.

$$y_i = r_i + \gamma \hat{Q}(s'_i, \bar{a}'_i), \quad \bar{a}'_i \sim \pi_\theta(\cdot | s'_i), \quad y_i = r_i \text{ at terminal states.}$$

Worked pattern. With $\gamma = 0.9$, transitions $(r_1, \hat{Q}(s'_1, \bar{a}'_1)) = (+2, 14), (+5, 9), (+8, 0 \text{ terminal})$:

$$y_1 = 2 + 0.9(14) = 14.6, \quad y_2 = 5 + 0.9(9) = 13.1, \quad y_3 = 8 + 0.9(0) = 8.0.$$

2.9 Why $\bar{a}$ is re-sampled from π_θ rather than read from the buffer

The E4 answer, in three parts.

- (i) The buffer action a_i came from an **old behaviour policy**. Using it would make $\hat{Q}$ an estimate of Q^{π_β} — the value of the policy that collected the data — and would improve that policy, not π_θ .
- (ii) Re-sampling $\bar{a}_i \sim \pi_\theta(\cdot | s_i)$ makes $\hat{Q}$ the value of the **current** policy, so $\nabla_\theta \log \pi_\theta(\bar{a}_i | s_i) \hat{Q}(s_i, \bar{a}_i)$ is a valid gradient estimate for π_θ .
- (iii) It **removes the need for importance weights entirely**. That is the “problem this solves”: there is no distribution mismatch left to correct, because you generated the action from the correct distribution yourself.

The slide’s own phrasing for the same point: *the current policy’s actions are likely better than the past policy’s actions.*

Note also. The slides drop the baseline at this stage and use $\hat{Q}$ rather than $\hat{A}$. This raises variance, but it is acceptable precisely because the replay buffer supplies far more data than an on-policy batch would.

Carry the tension forward — this earns marks in Lecture 08. Re-sampling $\bar{a}' \sim \pi_\theta$ is exactly the mechanism that **queries $\hat{Q}$ at out-of-distribution actions**. Online this is harmless: the agent will eventually try the action and reality will correct the estimate. **Offline it is fatal**, because nothing ever refutes the estimate. Mentioning this link when answering either question is likely to be the difference between partial and full marks.

3 Part III — Imitation Learning and Offline RL (Lecture 08)

This is the densest lecture and the practice paper gives it three scenarios. Budget the most study time here.

3.1 Imitation learning: the setting

What it is. You are given trajectories collected by an expert, $D = \{(s_1, a_1, \dots, s_T)\}$, sampled from some unknown policy π_{expert} . The goal is a policy π_θ that performs at the level of the expert by mimicking it. There are no rewards.

Plain English. Learn to drive by watching a good driver, without ever being told what “good” means numerically.

What problem it solves. Reward design. Specifying a reward function that actually produces the behaviour you want is famously hard; demonstrations sidestep it entirely. It also converts RL into supervised learning, which is far more stable and better understood.

What it cannot solve. It can never outperform the demonstrator. It also does no reasoning about outcomes or dynamics, and it breaks if the expert has different degrees of freedom from the learner (a human hand versus a two-finger gripper), or if demonstrations cannot be provided at all.

3.2 Behavioural cloning version 0: ℓ_2 regression, and the mean-seeking failure

The method. For a deterministic policy $\hat{a} = \pi_\theta(s)$, do supervised regression onto the expert’s actions:

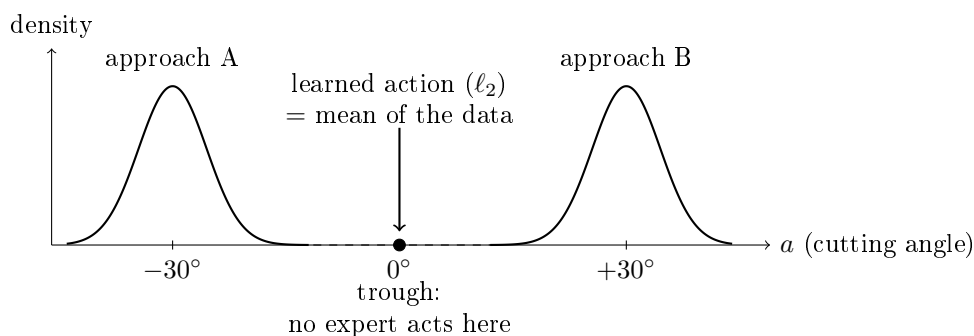
$$\min_{\theta} \frac{1}{|D|} \sum_{(s,a) \in D} \|a - \hat{a}\|^2.$$

What goes wrong — state the mechanism, not just the symptom. Squared error is minimised by the **conditional mean** $\mathbb{E}[a \mid s]$. If the expert data at a state is multimodal, the conditional mean is an action **no expert ever took**, sitting in the *trough* between the modes.

The canonical instance. Surgeons are equally split between approach A at $a = -30^\circ$ and approach B at $a = +30^\circ$. The learned action is $a = 0^\circ$.

The danger argument must be domain-specific, not generic. 0° is **not a compromise between two valid strategies**. It is a third action that lies precisely where the data density is lowest. On a highway with an obstacle, “swerve left” and “swerve right” are both valid and their average drives straight into the obstacle. In surgery, it cuts where neither approach cuts — into tissue that both experts deliberately avoided.

The sketch (marks are allocated to it — draw it).



Because the modes are far apart relative to their width, the combined (mixture) density is indistinguishable from the two bumps themselves everywhere except in the trough, where it is essentially zero — draw it as a dashed envelope joining the two bumps along the axis.

The quantitative version of the same point. If the true density is a two-component mixture, evaluating it at the mean gives a number essentially equal to zero. With $w_1 = w_2 = 0.5$, $\mu_1 = -30$, $\mu_2 = +30$, $\sigma_1 = \sigma_2 = 2$:

$$p(a \mid s) = 0.5\mathcal{N}(a; -30, 4) + 0.5\mathcal{N}(a; +30, 4), \quad p(0 \mid s) = 0.5(0) + 0.5(0) = \mathbf{0.000}.$$

(Watch the notation: in $\mathcal{N}(0; -30, 4)$ the third argument is the *variance*, consistent with $\sigma = 2$. The exact value is about 3×10^{-50} , i.e. 0.000 to three decimals.) The marks are for writing the mixture with the numbers substituted and for saying that $a = 0$ falls in the trough between the modes.

Advantages of ℓ_2 BC anyway. Trivially simple, stable, fast, and perfectly adequate when the expert is unimodal — which is common in low-level control such as joint-torque tracking.

When not to use it. Whenever multiple distinct valid strategies exist at the same state: navigation around obstacles, grasp selection, multi-route driving, any task with symmetry.

The counterfactual (worth stating). If you had used ℓ_1 instead of ℓ_2 you would learn the conditional *median*, not the mean. For a symmetric bimodal distribution the median is also 0° , so this does *not* fix the problem. The issue is not the choice of norm — it is that a deterministic policy can only output *one* action, so no regression loss whatsoever can represent two modes. That is why the fix must change the *output type* of the policy, not the loss.

3.3 Behavioural cloning version 1: expressive policies

The method. Train a *generative model* of the expert’s actions instead of a regressor:

$$\min_{\theta} -\mathbb{E}_{(s,a) \sim D} [\log \pi_{\theta}(a \mid s)],$$

i.e. maximise the log-probability of the demonstrated actions under an expressive conditional distribution $\pi_{\theta}(\cdot \mid s)$.

Why this fixes the problem. Maximum likelihood with an expressive distribution can place probability mass on *both* modes and near-zero mass in the trough. Regression cannot, because its output is a single point.

The trap the slides warn about explicitly. *The expressive power of the function is different from the expressive power of the neural network.* A very large network that outputs the mean of a single Gaussian is still a unimodal policy. Adding parameters does not add modes. You must change the *distribution class*, not the network capacity.

3.3.1 Representation (a): Gaussian Mixture Model

Mechanism. The network takes s and outputs the parameters of a K -component mixture, $\{w_k(s), \mu_k(s), \Sigma_k(s)\}_{k=1}^K$:

$$p(a_i \mid s_i) = \sum_{k=1}^K w_k(s_i) \mathcal{N}(a_i \mid \mu_k(s_i), \Sigma_k(s_i)),$$

$$\mathcal{L}(\theta) = - \sum_{i=1}^N \log \left[\sum_{k=1}^K w_k(s_i) \mathcal{N}(a_i \mid \mu_k(s_i), \Sigma_k(s_i)) \right].$$

Plain English. “There are K things an expert might do here; predict where each of them is, how wide it is, and how often it is chosen.” Multimodality is *explicit*: you can point at the components.

Advantages. Interpretable; cheap to sample (pick a component, draw a Gaussian); exact likelihood, so the training objective is the true MLE.

Disadvantages / what it cannot solve. K must be chosen *in advance*, and the right K is state-dependent in general. It scales badly to high-dimensional actions, because each component needs a full covariance ($O(D^2)$ parameters) or is crippled by a diagonal assumption. Training is prone to mode collapse, where several components pile onto the same mode. And it does not model dependencies *between action dimensions* any better than a single Gaussian does, unless the covariances are full.

When to use it. Low-dimensional action spaces with a small, known number of strategies.

3.3.2 Representation (b): autoregressive policies

Mechanism. Factorise the action across its *dimensions* using the chain rule:

$$\pi_{\theta}(a \mid s) = \prod_{d=1}^D \pi_{\theta}(a_d \mid s, a_{<d}), \quad a_{<d} = (a_1, \dots, a_{d-1}),$$

and generate sequentially: $s \rightarrow a_1 \rightarrow a_2 \rightarrow \dots \rightarrow a_D$, each $a_d \sim \pi_{\theta}(\cdot \mid s, a_{1:d-1})$. The loss is $\mathcal{L}(\theta) = -\sum_i \sum_d \log \pi_{\theta}(a_{i,d} \mid s_i, a_{i,1:d-1})$.

Plain English. Decide the components of the action one at a time, each one conditioned on the ones already chosen — like writing a sentence word by word. Once you have committed to “swerve left” in the first dimension, the remaining dimensions are chosen consistently with that.

Its distinct advantage — this is the axis a “compare” question targets. It **explicitly models dependencies between action dimensions**. This is a *different* axis from GMM multimodality: a GMM with K components represents K blobs; an autoregressive model represents arbitrary joint structure without ever fixing a number of modes.

Disadvantages. Sampling is sequential, so it costs D forward passes; the result depends on the (arbitrary) ordering of dimensions; and errors in early dimensions propagate into later ones.

Named instance: Decision Transformer. A sequence model trained to predict actions from a history of (return-to-go, state, action) tokens. If asked how it differs from actor-critic: it is **conditional sequence generation, not value estimation**. There is no critic, no Bellman backup and no policy-improvement step — the desired return is supplied as a conditioning token at inference and the model generates the action that historically accompanied that return.

3.3.3 Representation (c): diffusion / flow policies

Mechanism (exactly as on the slide). *Training:*

1. Sample a datapoint $x_1 \sim D$ and noise $x_0 \sim \mathcal{N}(0, 1)$.
2. Sample $t \sim p(t)$.
3. Interpolate linearly: $x_t = tx_1 + (1 - t)x_0$.
4. Minimise $\|v_{\theta}(x_t, t) - (x_1 - x_0)\|^2$.

Inference:

1. Sample noise $x_0 \sim \mathcal{N}(0, 1)$.

2. For $t \in \{0, \delta, 2\delta, \dots, 1 - \delta\}$: $x_{t+\delta} = x_t + v_\theta(x_t, t) \delta$.
3. Return x_1 .

For imitation learning, v_θ is conditioned on the state s_t , so it is a **state-conditional vector field** that transports noise onto expert actions.

The three-sentence answer the exam asks for. (1) Start from a sample of Gaussian noise. (2) Repeatedly take small Euler steps along the learned, state-conditioned velocity field $x_{t+\delta} = x_t + v_\theta(x_t, t) \delta$. (3) The endpoint x_1 is the action — and because different noise draws are transported to different places, different draws land in different modes.

Precision note that earns the extra mark. The slide is labelled “Diffusion Policy”, but the procedure given — linear interpolation $x_t = tx_1 + (1 - t)x_0$, regression onto the velocity $x_1 - x_0$, Euler integration — is **flow matching / rectified flow**, not DDPM. DDPM predicts the noise added at each step and samples by a reverse stochastic process. Answer using the slide’s procedure, but add half a sentence noting that the mechanism is a *learned vector field* rather than iterative denoising.

Advantages. Multimodality is implicit and unbounded — no K to choose. Scales well to high-dimensional and temporally-extended actions (action chunks). Currently the strongest empirical choice for visuomotor robot policies.

Disadvantages. Inference requires many sequential network evaluations, which is a real problem for high-frequency control. No exact likelihood. Sensitive to the number of integration steps δ . It is harder to inspect than a GMM — you cannot point at “the modes”.

3.3.4 Choosing between the three

	Mechanism	Use when	Avoid when
GMM	Explicit finite set of modes output per state	Low-dimensional actions; few, known strategies; interpretability matters	High-dimensional actions; unknown or state-varying number of modes
Autoregressive	Chain-rule factorisation across action <i>dimensions</i>	Dimensions are strongly coupled; discrete or tokenised actions; sequence models available	Latency-critical control (needs D sequential passes)
Diffusion / flow	Learned state-conditioned velocity field transporting noise to data	High-dimensional, continuous, richly multimodal actions; action chunking	Very high control frequency; when exact likelihoods are needed

3.4 Compounding error and distribution shift

What it is. The root cause: supervised learning assumes i.i.d. data, and **imitation learning violates that assumption**, because the policy’s own predictions determine the next state. Start the answer here, not at “errors compound”.

The mechanism, as a feedback loop.

1. The learner makes a small prediction error, so the next state differs slightly from any state the expert visited.

2. At that state the policy has *no training data*, so its error is larger and essentially arbitrary.
3. That larger error moves it further off the expert’s state distribution.
4. Errors therefore **grow with the horizon** rather than staying bounded — classically, quadratically in T rather than linearly.

Plain English. A driver who drifts one metre from the centre line has never seen a recording of a car one metre off the centre line, because the expert never got there. So it has no idea how to steer back — and drifts two metres. Then four.

What it means for the two obvious “fixes”. Collecting a lot more expert data and hoping for the best does not address the loop: more data on the *expert’s* distribution still leaves the learner’s off-distribution states unlabelled. The problem is not the *amount* of data; it is the *distribution* of the data.

3.5 DAgger (Dataset Aggregation)

The algorithm.

1. Roll out the learned policy π_θ to get $s'_1, a'_1, \dots, s'_T$.
2. Query the expert for the correct action at each visited state, $a^* \sim \pi_{\text{expert}}(\cdot \mid s')$.
3. Aggregate: $D \leftarrow D \cup \{(s', a^*)\}$.
4. Retrain: $\min_\theta L(\pi_\theta, D)$. Repeat.

The one-line update rule (a question demands it explicitly).

$$D \leftarrow D \cup \{(s', a^*)\} \quad \text{where} \quad s' \sim \pi_\theta, \quad a^* \sim \pi_{\text{expert}}(\cdot \mid s').$$

Make both sources explicit: **states from the learner, labels from the expert**. That is the entire idea.

Why it works — the insight is not “more data”. It is **data from the right distribution**. DAgger changes *where* the training states come from: instead of only expert states, it collects states the learner actually visits, so the training distribution converges to the deployment distribution and the feedback loop is closed.

What it cannot solve — the boundary. DAgger requires an **interactive expert available throughout training**. A surgeon must sit and label every state the robot wanders into; a submarine crew cannot be consulted about a trench the vehicle has never entered. In precisely the scenarios the exam chooses, that assumption is what fails. A question asking “how would DAgger have prevented this” invites a follow-up asking why DAgger was not used — have the answer ready.

Other costs. Querying an expert on states the learner reached can be dangerous or meaningless (asking a surgeon what to do having already cut the wrong thing). It also still cannot exceed the expert.

3.6 Why offline RL?

Online versus offline. Online RL alternates collecting data and updating the policy. Offline RL is given a *static* dataset and must train the policy on it, with no further interaction.

Why offline may be preferable (cheap marks — list them).

- Leverage datasets already collected by people or existing systems.
- Online data collection may be **risky or unsafe** — this is why the exam scenarios are a surgical robot and a submarine.
- Reuse data across previous experiments, projects, robots and institutions rather than re-collecting it.
- A blend is possible: **offline pre-training** → **online fine-tuning**, which gives a safe initialisation and then a limited amount of online correction of the value errors offline training could not detect.

Formal setting. Given $D = \{(s, a, s', r)\}$ sampled from some unknown *behaviour policy* π_β — possibly a *mixture* of policies — with $p_{\pi_\beta}(\tau) = p(s_1) \prod_{t=1}^T \pi_\beta(a_t | s_t) p(s_{t+1} | s_t, a_t)$, maximise $\mathbb{E}_{p_{\pi_\theta}}[\sum_t r(s_t, a_t)]$ — i.e. the expectation under the *learned* policy, which is not the distribution you have data from. That mismatch is the whole difficulty.

3.7 Offline RL versus imitation learning: the stitching argument

The two claims that define the field.

1. **Imitation cannot outperform the policy that collected the data; offline RL can.**
The mechanism is reward labels plus stitching.
2. **Stitching requires bootstrapping.** This is the load-bearing point.

The setup. The dataset contains a good behaviour $s_1 \rightarrow s_3$ and a separate good behaviour $s_7 \rightarrow s_9$, but no single trajectory going $s_1 \rightarrow s_9$. Can the agent learn to do it?

The value-propagation chain — structure your answer this way. Take the submarine version: $\tau_3 = \text{Surface} \rightarrow \text{Trench} \rightarrow \text{Target}$ with return +20, and $\tau_2 = \text{Deep} \rightarrow \text{Trench} \rightarrow \text{Fail}$ with return -8.

1. From τ_3 , TD learning assigns Trench a *high* value: $Q(\text{Trench}, \text{correct action})$ backs up from the +20 at Target, so $V(\text{Trench})$ becomes large.
2. τ_2 contains the transition Deep $\rightarrow$ Trench. The TD target for that transition is $r + \gamma V(\text{Trench})$ — it uses the *learned* value of Trench, **not** the return actually observed in τ_2 .
3. Because $V(\text{Trench})$ is high — learned from a *different* trajectory — the transition Deep $\rightarrow$ Trench receives a high value *despite* τ_2 ending in failure. The failure in τ_2 is attributed to what happened *after* Trench, not to arriving there.
4. **This works only with TD / dynamic programming.** Monte Carlo would credit the Deep $\rightarrow$ Trench transition with τ_2 's actual observed return of -8 and would learn the *opposite* conclusion. The slide asks exactly this (“Does it depend on whether you use TD or MC?”): the answer is **yes**, and the reason is bootstrapping.

Plain English. Bootstrapping lets information flow *between* trajectories, because the target for one transition is a value that some other trajectory established. Monte Carlo keeps each trajectory's information sealed inside itself.

What stitching cannot do. It cannot invent transitions. If no trajectory in the dataset ever performs the action that leads out of Deep towards Trench, no amount of bootstrapping will discover it. Stitching recombines observed *transitions*; it does not explore.

3.8 Why naive off-policy Q-learning fails offline: the OOD chain

The objective in question. The off-policy critic update (as in SAC):

$$\min_{\phi} \sum_{(s,a,s') \sim D} \left(\hat{Q}_{\phi}^{\pi_{\theta}}(s,a) - (r(s,a) + \gamma \mathbb{E}_{a' \sim \pi_{\theta}(\cdot|s')} [\hat{Q}_{\phi}^{\pi_{\theta}}(s',a')]) \right)^2.$$

Reproduce the causal sequence, not just the conclusion.

1. The critic target contains $\mathbb{E}_{a' \sim \pi_{\theta}(\cdot|s')} [\hat{Q}(s',a')]$, and a' is drawn from the *current policy* — it need not appear in D at all.
2. $\hat{Q}$ is **unreliable on out-of-distribution actions**: nothing in the data constrains its value there, so it is essentially the arbitrary output of a randomly initialised function approximator.
3. The policy update **actively seeks** the actions where $\hat{Q}$ is over-optimistic, because the policy is *maximising* $\hat{Q}$. This is not bad luck — it is the optimiser doing its job against a flawed proxy.
4. Those inflated values enter the *next* TD target, so the overestimation propagates and compounds. Values become substantially overestimated.
5. **Nothing corrects the error, because there is no environment interaction.** The learned policy therefore deviates arbitrarily far from the behaviour policy with no way back.

Step 5 is what makes this an offline problem — say so explicitly. Online, the agent tries the over-valued action, observes the true reward, and the estimate is refuted within a few episodes. Offline, the loop is closed: the error that caused the policy to choose the action is the same error that will never be tested. An answer that describes overestimation without explaining why the offline setting cannot self-correct is incomplete.

Connect it. This is the same structure as reward hacking under RLHF and as goal-classifier exploitation in Lecture 09: **optimising a learned proxy makes the optimiser seek out the proxy's errors**. Naming that connection costs one sentence.

3.9 Filtered behavioural cloning

The method.

1. Rank trajectories by return $r(\tau) = \sum_{(s_t, a_t) \in \tau} r(s_t, a_t)$.
2. Filter to the top $k\%$: $\tilde{D} = \{\tau \mid r(\tau) > \eta\}$.
3. Imitate the filtered dataset: $\max_{\theta} \sum_{(s,a) \in \tilde{D}} \log \pi_{\theta}(a \mid s)$.

Plain English. “We have reward labels, so let us just copy the episodes that went well.” It is the simplest possible use of reward information in an offline setting.

Advantages. Trivial to implement; completely avoids OOD actions (it only ever trains on dataset actions); no value function; no hyperparameters beyond η ; robust.

Two traps the exam sets.

- (i) **Filtering is at the trajectory level.** All state–action pairs from a kept trajectory survive, including the bad actions inside a mostly-good trajectory. There is no per-action credit assignment.
- (ii) **Coverage.** If only one kept trajectory passes through a state, the learned policy at that state is fully determined by that single sample, with *no evidence whatsoever* that the action taken there was the best available. That is the intended criticism.

What it cannot do. It cannot stitch — there is no bootstrapping anywhere in it. It also discards most of the dataset, which is wasteful when data is the scarce resource, and it inherits a ceiling: the best trajectory in the data.

Worked pattern (submarine). With $\eta = +5$ and returns $\tau_1 = +12$, $\tau_2 = -8$, $\tau_3 = +20$, $\tau_4 = +7$, $\tau_5 = -3$, the kept trajectories are τ_1, τ_3, τ_4 . Trench appears in the filtered set only via τ_3 (Surface $\rightarrow$ Trench $\rightarrow$ Target), so at Trench the robot learns exactly the one action τ_3 took — “head for Target” — with no evidence it is optimal, and no knowledge of the Trench $\rightarrow$ Shallow $\rightarrow$ Fail behaviour recorded in τ_5 , which was filtered out.

3.10 Weighted imitation learning and AWR

The idea. Rather than a hard keep/discard filter, weight each transition by *how good the action was*, measured by the advantage:

$$\theta \leftarrow \arg \max_{\theta} \mathbb{E}_{s,a \sim D} [\log \pi_{\theta}(a | s) \exp(A(s, a))].$$

Advantage-Weighted Regression, in full.

1. **Fit the value function** by regressing on *empirical Monte Carlo returns*:

$$\min_{\phi} \sum_{s_t \sim D} \left(\hat{V}_{\phi}^{\pi_{\beta}}(s_t) - \sum_{t'=t}^T r(s_{t'}, a_{t'}) \right)^2.$$

2. **Train the policy:**

$$\max_{\theta} \mathbb{E}_{(s_t, a_t) \sim D} \left[\log \pi_{\theta}(a_t | s_t) \exp \left(\frac{1}{\alpha} \left(\sum_{t'=t}^T r(s_{t'}, a_{t'}) - \hat{V}_{\phi}^{\pi_{\beta}}(s_t) \right) \right) \right].$$

Read it as weighted behaviour cloning. It is still maximum likelihood on *dataset* actions — the policy is never trained on an action nobody took — but each sample is weighted by $\exp(A/\alpha)$. Filtered BC is the special case where the weight is binary.

What α does — it is a *temperature*, not merely an overflow guard. Small α sharpens the weights, approaching an argmax over dataset actions, i.e. filtered BC. Large α flattens them towards uniform, i.e. plain BC. The slide’s phrasing (“a hyperparameter to avoid exploding due to the exponential”) is the numerical half; the dial is the conceptual half, and a question asking “what does α control” wants the dial.

Why Monte Carlo for $\hat{V}$? Because it **avoids querying or training on any OOD action**. An MC return is computed purely from rewards that were actually observed; no $\hat{Q}(s', a')$ is ever evaluated at an unseen a' . That is precisely the property TD sacrifices.

Trade-offs, exactly as the slides give them.

Monte Carlo estimation: for	Monte Carlo estimation: against
Simple	Noisy (high-variance returns)
Avoids querying or training on any OOD action	$\hat{A}^{\pi_{\beta}}$ is the advantage for the <i>weaker</i> behaviour policy, not for π_{θ}

The interpretation trap (this is engineered into the paper). A high AWR weight on an action seen very few times is **not trustworthy**. $\hat{Q}$ for a rarely-seen action is a high-variance estimate, and AWR *exponentiates* it, amplifying the error. The full-marks sentence: *AWR trusts $\hat{Q}$ unconditionally and has no uncertainty penalty — $\hat{Q}$ is only as good as its data support*. The presence of a “times seen in data” column in a question is the signal that this is being tested, not the arithmetic.

Worked pattern (logistics drone). $\hat{V}(s_D) = 11, \alpha = 1$:

$$\text{Fly North: } e^{15-11} = e^4 \approx 54.60, \quad \text{Fly South: } e^{6-11} = e^{-5} \approx 0.0067,$$

$$\text{Fly East: } e^{10-11} = e^{-1} \approx 0.368.$$

Normalised, these are $\approx 99.3\%, 0.01\%, 0.67\%$. AWR overwhelmingly favours **Fly North**, which is also the most frequent action in the data (80 occurrences) — so here the weight and the data support agree. Fly East, seen only 5 times, gets a weight ~ 55 times smaller than North’s, so the ranking happens to be safe — but the reasoning must be stated: had Fly East’s $\hat{Q}$ been the largest, AWR would have chased it, because it has no mechanism for penalising the uncertainty of an estimate built from five samples.

The slide’s own question: what do you learn for a deterministic π_β ? With a deterministic behaviour policy there is exactly one action per state in the data, so the empirical return equals $\hat{V}^{\pi_\beta}(s)$ for that sample, every advantage is 0, every weight is $e^0 = 1$, and **AWR degenerates to plain behaviour cloning**. The deeper reading: weighted imitation can only rank actions it has seen *alternatives* to. With no alternatives there is nothing to weight.

When to use AWR. Offline datasets from a mediocre but broad behaviour policy, where safety matters more than squeezing out the last increment of performance, and where trajectories are short enough that MC returns are not hopelessly noisy. **When not to.** Long horizons (MC variance explodes), or when you need to substantially exceed π_β .

3.11 Advantage-Weighted Actor–Critic (AWAC)

What changes. Replace the Monte Carlo value estimate with a TD-learned Q -function:

1. **Estimate the Q -function:**

$$\min_{\phi} \mathbb{E}_{(s,a,r,s') \sim D} \left[\left(\hat{Q}_{\phi}^{\pi_{\theta}}(s,a) - (r + \gamma \mathbb{E}_{a' \sim \pi_{\theta}(\cdot | s')} [\hat{Q}_{\phi}^{\pi_{\theta}}(s',a')]) \right)^2 \right].$$

2. **Update the policy with AWR:**

$$\theta \leftarrow \arg \max_{\theta} \mathbb{E}_{s,a \sim D} \left[\log \pi_{\theta}(a | s) \exp(\hat{A}^{\pi_{\theta}}(s,a)) \right],$$

$$\text{where } \hat{A}^{\pi_{\theta}}(s,a) = \hat{Q}_{\phi}^{\pi_{\theta}}(s,a) - \mathbb{E}_{\bar{a} \sim \pi_{\theta}(\cdot | s)} [\hat{Q}_{\phi}^{\pi_{\theta}}(s,\bar{a})].$$

The advantage/disadvantage pair the exam wants (state exactly one of each).

- **Advantage.** The critic estimates $\hat{Q}^{\pi_{\theta}}$ for the *current* policy rather than $\hat{Q}^{\pi_{\beta}}$ for the weaker behaviour policy, and TD is lower-variance than Monte Carlo — so the advantage estimates are better and the method can genuinely improve *beyond* the behaviour policy. It also still trains the policy only on dataset actions.
- **Disadvantage.** The TD target samples $a' \sim \pi_{\theta}(\cdot | s')$, so it **queries $\hat{Q}$ at out-of-distribution actions**, reopening exactly the overestimation problem that AWR structurally avoided. With a fixed offline dataset and no way to verify, that is dangerous.

The counterfactual. The slides note an alternative: sample $a' \sim D$ instead of from π_θ . That would restore safety — no OOD queries — but it would also make the critic estimate Q^{π_β} again, losing the very thing AWAC was introduced to gain. This is the trade in miniature, and stating it demonstrates that you understand the design rather than the pseudocode.

3.12 Expectile regression

The goal. We want TD-quality advantage estimates *without* querying Q on OOD actions. The idea: instead of a mean, estimate a higher expectile of the distribution of Q values over the actions the data contains — “can we use the 70th percentile instead of the mean?”

The loss.

$$\ell_2^\lambda(x) = \begin{cases} (1 - \lambda)x^2, & x < 0, \\ \lambda x^2, & \text{otherwise.} \end{cases}$$

Understand the mechanism, not the label. The asymmetric weight makes *one direction of error cheaper*, so the minimiser slides towards the side it is cheap to sit on. $\lambda = 0.5$ recovers ordinary least squares and hence the mean. If you hold the mechanism, you can derive the direction for whatever convention a paper hands you — which matters here, because the lecture slide and the standard IQL paper write the residual in *opposite* orders (see Section 6).

Worked numbers (I verified these by direct minimisation). Take the values $Q \in \{15, 6, 10\}$, mean 10.33.

	$\lambda = 0.1$	$\lambda = 0.5$	$\lambda = 0.8$
Slide convention: $\ell_2^\lambda(V - \hat{Q})$	$\hat{V} = 13.73$	10.33	8.17
IQL-paper convention: $\ell_2^\tau(\hat{Q} - V)$	$\hat{V} = 7.18$	10.33	12.67

Read the table as the proof of the mechanism: under the slide’s ordering, $x = V - Q < 0$ means V sits *below* Q , and small λ makes that case expensive (weight $1 - \lambda$), pushing $\hat{V}$ *up*. Under the paper’s ordering everything flips. Same mechanism, opposite label.

3.13 Implicit Q-Learning (IQL)

The full algorithm, in order.

1. **Fit $\hat{V}$ with the expectile loss:** $\hat{V}(s) \leftarrow \arg \min_V \mathbb{E}_{(s,a) \sim D} [\ell_2^\lambda(V(s) - \hat{Q}(s,a))]$ (the slide instructs: using small $\lambda < 0.5$).
2. **Update Q with an ordinary MSE loss:** $\hat{Q}(s,a) \leftarrow \arg \min_Q \mathbb{E}_{(s,a,s') \sim D} \left[(Q(s,a) - (r + \gamma \hat{V}(s')))^2 \right]$.
3. **Extract the policy with AWR:** $\hat{\pi} \leftarrow \arg \max_\pi \mathbb{E}_{s,a \sim D} \left[\log \pi(a | s) \exp\left(\frac{1}{\alpha}(\hat{Q}(s,a) - \hat{V}(s))\right) \right]$.

Why IQL never touches OOD actions — the key sentence, phrased for the E4 question. Standard Q-learning takes its bootstrap target from $\max_{a'} \hat{Q}(s', a')$ or $\mathbb{E}_{a' \sim \pi_\theta} [\hat{Q}(s', a')]$, *both of which require evaluating $\hat{Q}$ at actions that need not appear in the data*. IQL replaces that entire operation with $\hat{V}(s')$, and $\hat{V}$ is fit by **expectile regression over only the (s,a) pairs present in D** . The upper expectile approximates a maximum *restricted to the action support of the data*. Therefore no OOD action is ever queried, anywhere in the algorithm. Notice that step 2 also only ever evaluates Q at dataset (s,a) pairs, and step 3 only ever takes $\log \pi(a | s)$ for $a \in D$ — the safety property holds for all three steps, not just one.

Why an *upper* expectile is the useful one. $\hat{V}(s)$ should reflect **the best outcome achievable from s using actions the data supports**, not the average over the good and the bad behaviour recorded at s . In the submarine dataset, the mean over actions at Trench would mix τ_3 ’s successful route to Target with τ_5 ’s route to Fail, dragging $V(\text{Trench})$ down — and $V(\text{Trench})$ is precisely the quantity that stitching depends on. The upper expectile keeps the value at what the best recorded route achieved.

What IQL cannot solve. It is **conservative by construction**: it can only ever recombine actions present in the data, so it cannot discover a genuinely new action, however obviously good. It adds a hyperparameter λ (plus AWR’s α). And a high expectile with scarce data at a state is still an optimistic estimate over a small, noisy sample — pushed far enough, the expectile is a maximum over noise.

Counterfactual — what if step 1 used a plain MSE? Then $\hat{V}(s) = \mathbb{E}_{a \sim \pi_\beta}[\hat{Q}(s, a)]$, i.e. the value of the *behaviour* policy. Step 2 would then perform policy *evaluation* of π_β rather than improvement, and the whole method would collapse to something close to AWR-with-TD: safe, but with no mechanism for exceeding π_β . The expectile is what smuggles a max into the algorithm without ever writing one.

3.14 The four offline methods as one progression

The unifying insight (this is a full-marks answer to any “explain the relationship” question). **All four are weighted behaviour cloning.** None of them ever trains the policy on an action outside the dataset: the policy objective is always $\log \pi_\theta(a \mid s)$ for $a \in D$, with a weight in front. What differs is *only how the weight is computed*, and every row of the table below is a statement about how much machinery you are willing to run to get a better weight.

	Weight / value signal	OOD queries?	The price
Filtered BC	Binary; thresholded trajectory return	None	Throws away most of the data; keeps bad actions from good trajectories; no per-action credit; cannot stitch
AWR	$\exp(A/\alpha)$ with Monte Carlo $\hat{V}^{\pi_\beta}$	None	MC is noisy; the advantage is for the <i>weaker</i> behaviour policy, not for π_θ
AWAC	$\exp(A/\alpha)$ with TD $\hat{Q}^{\pi_\theta}$, $a' \sim \pi_\theta$	Yes , in the critic	Reopens OOD overestimation, which offline cannot be corrected
IQL	$\exp(A/\alpha)$ with expectile $\hat{V}$ and MSE $\hat{Q}$	None	Extra hyperparameter λ ; conservative by construction; still bounded by the data’s action support

The axis to say out loud. *How good is the value estimate, and what did it cost in OOD exposure?* Filtered BC pays nothing and gets almost nothing. AWR pays noise. AWAC pays safety. IQL pays a hyperparameter and some conservatism, and is the only one that buys TD-quality values without selling safety — which is why it is the one you would deploy on the submarine.

4 Part IV — Reward Learning and RLHF (Lecture 09)

Only sixteen slides, but it carries two full scenarios and part of a third. This is the highest marks-per-page section of the paper.

4.1 Where does the reward come from?

The framing question. Direct imitation learning mimics the expert's *actions*. That has three named limitations on the slide:

- no reasoning about outcomes or dynamics — the learner copies the motion, not the intent;
- the expert may have *different degrees of freedom* (a human five-fingered hand demonstrating to a two-finger gripper);
- demonstrations may be impossible to provide at all.

So: can we instead reason about *what the expert is trying to achieve*, and learn a *reward*? A learned reward is reusable, composable, and can in principle be optimised past the expert's own competence.

4.2 Goal classifiers

The method.

1. Collect examples of successful and unsuccessful states — states inside and outside the goal set $\mathcal{G}$.
2. Train a binary classifier with inputs s_i and labels $\mathbf{1}(s_i \in \mathcal{G})$.
3. Run RL using the classifier's output $C_\phi(s)$ as the reward.

Plain English. Rather than writing a reward function, show the system pictures of success and let it learn to recognise them; then let RL chase that recogniser.

Advantages. Requires no reward engineering, no demonstrations, and no expert who can perform the task — only someone who can *recognise* the goal. Labelling states is far cheaper than teleoperating a robot.

What goes wrong — name it and give the mechanism. The RL algorithm seeks states the classifier *scores* highly, which is not the same as states that *are* goals. It will find states the classifier was never trained on, where the classifier's output is meaningless, and exploit them. The names to use: **exploiting the classifier's weaknesses**, **adversarial exploitation of a learned reward**, **reward hacking**. The canonical instance: a cup placed *upside-down* on the coaster scoring 0.91. It scores highly because it is visually close to the positives and unlike anything in the negatives — and it was never in either training set.

The fix. Close the loop: make the classifier learn from what the agent actually does.

1. Collect an initial set of successful states D^+ and unsuccessful states D^- .
2. Update the classifier using D^+ and D^- , **balancing the batches 50/50**.
3. Collect experience $s_t, a_t, \dots$ using the policy π .
4. Update the policy using the classifier-based reward.
5. Add *all* visited states to the negatives: $D^- \leftarrow D^- \cup \{s_t\}$. Go to step 2.

Why the fix works — the mechanism. It removes the profit from any state the agent has actually reached. Whatever exploit the policy discovers this round becomes a labelled negative next round, so its reward collapses. The agent cannot stay ahead of a classifier that is retrained on its own discoveries. This is a general and reusable pattern: the way to stop an optimiser exploiting a learned proxy is to keep retraining the proxy on the optimiser's current distribution.

The apparent paradox, and the answer the exam wants. True goal states get visited during RL, so they end up in D^- as well. Why does the classifier still reward the goal?

Because the batches are **balanced 50/50**. A true goal state appears in D^+ (the original positives plus goals reached during RL) *and* in D^- (the rollout sweep). But D^+ is small and D^- is large; rebalancing to 50/50 **upweights each positive example** relative to each negative one. For a state appearing on both sides, the effective evidence is at least even, so the optimal classifier output is $p \geq 0.5$. Non-goal states appear *only* in D^- and are driven towards 0. The goal therefore remains the highest-reward state even though it is nominally also a negative.

A question that raises this worry is asking for the balancing argument specifically. An answer that says “it still works because goal states look different” scores nothing.

Worked counting pattern. After 500 RL steps the agent has visited 500 states, of which 30 are true goal states. Then *all* 500 visited states go into D^- , so $|D^-| = 500$ (plus whatever was there before), and $|D^+| = 50 + 30 = 80$. Note that the 30 goal states are counted on *both* sides — that is the entire point of part (ii).

What goal classifiers cannot do. They are **binary**. They cannot rank two successes, cannot express “this is a better grasp than that one”, and give no shaping signal — the reward is flat everywhere except at the goal, so exploration is as hard as with a sparse hand-written reward.

4.3 Learning rewards from human preferences

Why preferences rather than demonstrations or scores. Relative comparisons are **easier for humans to provide**: judging which of two rollouts is better requires no skill at the task and no calibration. And the reward model only needs to be **discriminative**, not calibrated — it must order responses, not assign meaningful absolute numbers.

The model. A human says τ_w is better than τ_l (each may be a full or partial rollout). We want a reward with

$$\sum_{(s,a) \in \tau_w} r_\theta(s,a) > \sum_{(s,a) \in \tau_l} r_\theta(s,a),$$

and we model the probability of a preference by the **Bradley–Terry** form

$$P(\tau_a \succ \tau_b) = \sigma(r_\theta(\tau_a) - r_\theta(\tau_b)),$$

maximising $\max_\theta \mathbb{E}_{\tau_w, \tau_l} [\log \sigma(r_\theta(\tau_w) - r_\theta(\tau_l))]$. In the LLM notation the reward model loss is

$$J_{RM}(\phi) = -\mathbb{E}_{(x, y_w, y_l) \sim D} \left[\log \sigma(RM_\phi(x, y_w) - RM_\phi(x, y_l)) \right].$$

The one fact to notice now, before DPO. The loss depends only on the *difference of the two rewards*. This is not an incidental algebraic feature — it is the hinge on which DPO turns. It also explains why the reward model is determined only up to an additive constant per prompt.

The learning algorithm.

1. Given a dataset $\{\tau_i\}$, sample batches of k trajectories and ask humans to *rank* them (for LLMs, all k share the same prompt).
2. Compute $r_\theta(\tau_1), \dots, r_\theta(\tau_k)$ under the current model.
3. For all $\binom{k}{2}$ pairs in the batch, compute $\nabla_\theta \mathbb{E}_{\tau_w, \tau_l} [\log \sigma(r_\theta(\tau_w) - r_\theta(\tau_l))]$.
4. Update θ ; repeat.

Note the slide’s remark: this can be done *inside the loop of online RL*. That is the structural fix for reward hacking — retraining the reward model on the current policy’s outputs keeps it valid on the distribution the policy is actually exploring.

Combinatorics. Ranking k items yields $\binom{k}{2}$ pairwise comparisons: $k = 6 \Rightarrow 15$. This is the data-efficiency argument for asking for a ranking rather than a single pairwise query. Do not lose this mark.

Compute patterns.

Loss for a pair: $RM(x, y_w) = 4.0, RM(x, y_l) = 1.5 \Rightarrow \Delta = 2.5, \sigma(2.5) \approx 0.924$,

$$J_{RM} = -\log(0.924) \approx \mathbf{0.079}.$$

Preference probability: $RM(x, y_A) = 3.2, RM(x, y_B) = 3.0 \Rightarrow \Delta = 0.2$,

$$P(y_A \succ y_B) = \sigma(0.2) \approx \mathbf{0.550}.$$

Read the numbers (E2). A loss of 0.079 is *low*: the model already ranks this pair correctly and confidently, so this example contributes almost no gradient. A probability of 0.550 is *barely above chance*: the model considers these two responses nearly equivalent — a 0.2 gap in reward is small on the logit scale.

What preference learning cannot solve. It cannot detect that the *annotators* are wrong, inconsistent, or systematically biased. It inherits whatever the labellers preferred, including sycophancy, verbosity and confident-sounding falsehood. And Bradley–Terry assumes preferences are transitive and generated by a single latent scalar — which real human populations do not satisfy.

4.4 The three-stage RLHF pipeline

Stage	Objective	Data / signal required
1. Large-scale pre-training	Next-token prediction (maximum likelihood on raw text)	Massive unlabelled text corpus, mixed quality
2. Supervised fine-tuning (SFT)	Maximum likelihood on demonstrations	Curated high-quality (prompt, response) pairs
3. RL from human feedback	Maximise RM_ϕ minus β -KL to p^{PT}	Human pairwise preference comparisons (x, y_w, y_l) , used first to train RM_ϕ

Trap. Stage 3’s data is *preferences*, not (prompt, response) pairs. The lecture’s subtitle for stage 3 repeats stage 2’s text; that is a typo. Do not copy it.

4.5 The RLHF objective and the KL penalty

The setup. You have a pre-trained (possibly instruction-fine-tuned) model $p^{PT}(y \mid x)$ and a reward model $RM_\phi(x, y)$ trained on human comparisons. Copy the model to get $p_\theta^{RL}(y \mid x)$ with trainable parameters θ , and optimise

$$\max_{\theta} \mathbb{E}_{\hat{y} \sim p_\theta^{RL}(\hat{y} \mid x)} \left[RM_\phi(x, \hat{y}) - \beta \log \frac{p_\theta^{RL}(\hat{y} \mid x)}{p^{PT}(\hat{y} \mid x)} \right].$$

What the KL term penalises. **Drift of the fine-tuned policy away from the pre-trained model’s distribution**, measured per response as a log-ratio. β is the *price* of that drift.

The two limits (a direct exam question).

- $\beta = 0$: pure reward maximisation with no anchor. The model exploits the reward model’s errors, produces degenerate high-scoring text, and loses fluency and general capability.
- $\beta = 100$: the penalty dominates, so $p_\theta^{RL} \approx p^{PT}$ — you get the pre-trained model back and essentially no learning from preferences occurs.

Reward hacking / over-optimisation — the named failure at moderate β with too many steps.

1. RM_ϕ was trained on a *finite* sample of comparisons drawn from a particular distribution of responses.
2. It is therefore accurate on that distribution and **unreliable off it**.
3. The RL policy is explicitly searching for the highest-scoring responses, which *drives it off that distribution* — it is optimising towards the reward model’s blind spots by construction.
4. The KL penalty works by keeping the policy in the region where the reward model is still valid. Too small a β , or too many steps, and the policy escapes that region.

Diagnosis must be two-sided. One sign alone does not establish over-optimisation; the pair does.

- **Sign 1:** the reward-model score keeps rising while human-judged quality falls — e.g. confident but factually wrong answers.
- **Sign 2:** outputs look qualitatively unlike the pre-trained model’s; the measured KL from p^{PT} is growing large. Often accompanied by degenerate stylistic tics and collapsing diversity.

Remedy: *increase β* . Also legitimate: early stopping, and retraining the reward model on fresh on-policy comparisons. Name the direction — “adjust β ” without a direction does not score.

Say the connection. This is structurally the **same failure** as OOD exploitation in offline RL and as goal-classifier exploitation. *Optimising a learned proxy makes the optimiser seek out the proxy’s errors*. The exam instantiates this idea in three separate scenarios and is testing whether you notice it is one idea appearing three times.

4.6 Direct Preference Optimization (DPO)

The derivation as a chain — a question can ask for any link.

1. The KL-regularised objective has a **closed-form optimum**:

$$p^*(\hat{y} \mid x) = \frac{1}{Z(x)} p^{PT}(\hat{y} \mid x) \exp\left(\frac{1}{\beta} RM(x, \hat{y})\right).$$

2. Rearranging for the reward:

$$RM(x, \hat{y}) = \beta \log \frac{p^*(\hat{y} \mid x)}{p^{PT}(\hat{y} \mid x)} + \beta \log Z(x).$$

3. This identity holds for an *arbitrary* language model, so define

$$RM_\theta(x, \hat{y}) = \beta \log \frac{p_\theta^{RL}(\hat{y} \mid x)}{p^{PT}(\hat{y} \mid x)} + \beta \log Z(x).$$

The policy is implicitly its own reward model. If you can say that sentence and justify it from the log-ratio identity, you have the topic.

4. Substitute into the Bradley–Terry loss. Only the *difference* $RM_\theta(x, y_w) - RM_\theta(x, y_l)$ appears, and $\beta \log Z(x)$ is identical in both terms because it depends on x alone — so it **cancels**:

$$\ell_{DPO} = -\log \sigma \left(\beta \log \frac{p_\theta^{RL}(y_w \mid x)}{p^{PT}(y_w \mid x)} - \beta \log \frac{p_\theta^{RL}(y_l \mid x)}{p^{PT}(y_l \mid x)} \right).$$

On $Z(x)$ — three things must be said.

- **What it is.** The partition function, i.e. the normalising constant for prompt x : a sum over the *entire space of possible responses*, $Z(x) = \sum_y p^{PT}(y \mid x) \exp(RM(x, y)/\beta)$. It is intractable, because that space is exponentially large in sequence length.

- **Why it cancels.** It is a function of *x only*, not of *y*. The DPO loss uses only the reward *difference* between two responses to the *same* prompt, so the term is subtracted from itself. This is what makes training tractable without ever computing Z .
- **The practical consequence** (the part most often missed). You can compute *relative* rewards — which of two responses is preferred — exactly. But the **absolute** reward of a single response y is known only up to the unknown constant $\beta \log Z(x)$. That is entirely sufficient for preference optimisation and insufficient for anything needing calibrated absolute scores (thresholding, filtering, cross-prompt comparison, best-of- n with an absolute cut-off).

Compute pattern. $\ell_{DPO} = -\log \sigma(\beta(\text{ratio}_w - \text{ratio}_l))$. With $\beta = 0.5$ and log-ratios 1.2 and 0.2:

$$\text{argument} = 0.5(1.2 - 0.2) = 0.5, \quad \sigma(0.5) \approx 0.622, \quad \ell_{DPO} = -\log(0.622) \approx \mathbf{0.475}.$$

Substitute inside-out and show each layer — the marks are for the layers.

The advantage over two-stage RLHF, in one sentence. DPO removes the separate reward-model training stage *and* the RL loop entirely, optimising the policy directly on preference pairs with a simple supervised-style classification loss — no sampling, no rollouts, no separate frozen RM_ϕ for the policy to hack, and therefore far cheaper and more stable.

What DPO cannot solve.

- It still needs human preference labels, and inherits every bias in them.
- It optimises against a *fixed* dataset of preferences, so it cannot re-query the annotator on the policy’s new outputs the way online RLHF with a retrained reward model can. It trades reward-model over-optimisation for *dataset* over-optimisation.
- It gives no usable absolute reward, as above.
- The implicit reward is only exactly equivalent to the RLHF optimum under the assumptions of the derivation (Bradley–Terry preferences, the same β , exact optimisation).

When to use which. DPO when you have a fixed high-quality preference dataset and want a cheap, stable, reproducible fine-tune. Two-stage RLHF when you can afford to keep collecting fresh comparisons on the current policy’s outputs, which is the structural defence against over-optimisation.

4.7 RLAIIF / Constitutional RL

What it is. Ask another language model “which of these responses is less harmful?” and use its answers as the preference labels, in place of (or alongside) human labels.

The justifying insight, in the five words the slides use: *critique is easier than generation*. A model that cannot reliably *produce* the best response can still reliably *judge* which of two responses is less harmful. That asymmetry is the entire licence for AI annotation, and a question asking for “the key insight from the slides” wants exactly this.

Practical advantages at internet scale (give two).

- **Throughput and cost:** essentially unlimited labelling volume at negligible marginal cost, scaling instantly to new content volumes.
- **Consistency:** a written policy or constitution is applied uniformly, with no annotator-to-annotator variance, no fatigue, and no drift over a long shift.
- (Also creditable: it avoids exposing human moderators to harmful content — a real occupational-health benefit for a content-moderation task.)

The unique risk — and note the question *filters for uniqueness*. A generic criticism (“AI labels may be wrong”) fails the filter, because human labels may also be wrong. The answer must be a property that belongs to AI annotation *and not* to human annotation:

The annotator’s biases are systematic and correlated across every single label. Human errors are noisy and partially independent, so they largely average out across a pool of annotators. A single model’s blind spots do not average out — the same blind spot is applied identically to every example. The reward model therefore inherits them wholesale, and the RL policy then optimises *directly against them*, with no independent ground truth anywhere in the loop to detect the drift.

Relatedly. There is no external check on the pipeline: the model being trained, the annotator, and often the base model are all from the same family, so the whole system can drift together with nothing outside it to notice.

4.8 Goal classifiers versus RLHF — the comparison table

Aspect	Goal classifier	RLHF (human preferences)
Type of human input	A set of labelled example <i>states</i> (success / failure), provided once up front	Pairwise <i>comparisons</i> between roll-outs or responses, provided on an on-going basis
Handles nuanced quality?	No — binary success/failure only; cannot rank two successes or express degree	Yes — relative preferences give a graded, continuous reward
Main failure mode	The agent exploits states the classifier was never trained on (adversarial exploitation of the classifier)	Reward hacking / over-optimisation of the learned reward model

The sentence that ties it together: the two failure modes are the *same phenomenon* at different scales — proxy exploitation.

5 Part V — The Threads Running Through All Four Lectures

5.1 Thread 1 — Every fix introduces a new failure

If you memorise one thing, memorise this chain. Each arrow is a fix; each parenthetical is the price.

The value-based / policy-gradient spine. Vanilla PG (unbiased, hopeless variance) → reward-to-go (partial credit assignment; still high variance) → baseline (removes the all-positive failure; still on-policy, still state-blind) → learned critic / advantage (state-relative credit; now *biased*) → GAE (a bias–variance dial; extra hyperparameter) → importance sampling (data reuse; weights explode with horizon) → KL constraint (contains the explosion; hard to enforce) → PPO clipping (cheap soft version; only valid *locally*) → replay buffer + $\hat{Q}$ with $\bar{a}' \sim \pi_\theta$ (full off-policy; **queries OOD actions**) → offline, this becomes fatal → AWR (no OOD, but MC-noisy and stuck at π_β) → AWAC (better values, but OOD returns) → IQL (expectile $\hat{V}$ removes OOD queries; conservative).

The imitation spine. ℓ_2 BC (mean-seeking, catastrophic on multimodal data) → MLE with expressive policies: GMM / autoregressive / diffusion (multimodality solved) → but distribution shift and compounding error remain → DAgger (fixes the data distribution; needs an *interactive expert*) → offline RL (uses rewards, can stitch and exceed the demonstrator; opens the OOD problem).

The reward spine. Hand-designed reward (hard to specify) → goal classifier (no engineering needed; the agent exploits it) → add visited states as negatives, 50/50 balanced (exploit removed; still binary) → human preferences (graded, easy to give; the reward model gets hacked) → KL penalty with β (contains the hacking; two-stage and expensive) → DPO (removes the RL loop and the separate reward model; still needs human labels, no absolute reward) → RLAIIF (unlimited labels; **annotator bias is systematic and uncorrectable**).

5.2 Thread 2 — Proxy exploitation is one idea appearing three times

	The proxy	Where it is invalid	The containment
Offline RL	$\hat{Q}(s, a)$	Out-of-distribution actions — no data constrains it there	Never query OOD actions (AWR, IQL); or stay near π_β
RLHF	$RM_\phi(x, y)$	Responses far from the comparison distribution it was trained on	KL penalty with β ; retrain RM_ϕ on-policy; early stop
Goal classifiers	$C_\phi(s)$	States never seen in D^+ or D^-	Add all visited states to D^- , rebalance 50/50, retrain

The one-sentence generalisation: *Optimising a learned proxy causes the optimiser to seek out the proxy’s errors, so every learned reward or value must be paired with a mechanism that either keeps the optimiser where the proxy is valid, or keeps the proxy valid where the optimiser goes.* Both containment strategies appear above — the first row uses the first, the third row uses the second, and RLHF uses both.

5.3 Thread 3 — Bias versus variance in bootstrapping

Estimator	Bias	Variance	Where it appears
Monte Carlo return	None	Highest	REINFORCE, reward-to-go, AWR's $\hat{V}$
n -step / GAE with λ near 1	Low	High	PPO with large λ
n -step / GAE with λ near 0	High	Low	PPO with small λ
One-step TD error δ_t	Highest	Lowest	Actor-critic, AWAC, IQL's Q step

The same dial appears as a *capability* question in offline RL: Monte Carlo cannot stitch, TD can. So the choice is not purely statistical — bootstrapping buys you the ability to combine information across trajectories, and pays for it in bias and (offline) in OOD exposure.

5.4 Thread 4 — Everything offline is weighted behaviour cloning

Filtered BC, AWR, AWAC and IQL all maximise $\mathbb{E}_{(s,a) \sim D} [w(s,a) \log \pi_\theta(a \mid s)]$ with a always drawn from the dataset. The weight w is binary for filtered BC and $\exp(A/\alpha)$ for the other three. The entire progression is a sequence of increasingly good ways to compute A — Monte Carlo, TD with $a' \sim \pi_\theta$, TD with an expectile $\hat{V}$ — and the entire cost accounting is about how much OOD exposure each one buys.

6 Errors and Ambiguities in the Source Materials

The paper’s cover instruction — “*if any information appears to be missing, state a reasonable assumption clearly and proceed*” — is not boilerplate. Several parts are genuinely under-specified or contradict the lecture slides. These are exactly where you should use that clause, and doing so *visibly* is what earns the marks.

6.1 (a) Scenario 1 Q3 contains a false premise — do not accept it

The manager asks why the gradient would *reduce* the log-probability of τ_2 ’s $t = 1$ action. Compute first. With $b = 4.5$, τ_2 ’s reward-to-go at $t = 1$ is 9, so the baselined value is $9 - 4.5 = +4.5 > 0$ — the gradient **increases** it. (Under a per-timestep baseline the conclusion is the same: the four $t = 1$ reward-to-go values are $\{11, 9, 0, -2\}$, whose mean is also 4.5.)

The full-marks answer has two moves. First, correct the premise with the arithmetic. Second, make the *deeper* point the question is really about: reward-to-go credits the $t = 1$ action with the +10 delivery that followed it, so a genuinely bad action (moving away from the shelf, $r_1 = -1$) is *reinforced* because a good outcome happened later. That is a **credit-assignment limitation of reward-to-go with a scalar baseline**, and it is precisely what a state-dependent advantage $A^\pi(s, a)$ from Lecture 07 exists to fix — A^π asks whether the action was better than what the policy usually does *at that state*, which reward-to-go cannot.

This part is engineered to catch students who reason from the narrative instead of the arithmetic. **Compute, then answer.**

6.2 (b) Scenario 6 Q4: the expectile convention is reversed relative to the standard

The lecture slide writes $\ell_2^\lambda(V(s) - \hat{Q}(s, a))$ and instructs “*using small $\lambda < 0.5$* ”. Under that formula the residual is $x = V - Q$, so $x < 0$ means V sits *below* Q , and that case carries weight $(1 - \lambda)$. Small λ therefore makes underestimation expensive and pushes $\hat{V}$ **up** — so under the slide’s formulation, *small λ* is the *upper* expectile.

The question, however, sets $\lambda = 0.8$ and asks whether $\hat{V}$ is pushed above or below the mean. As the verified table in the expectile section shows, under the slide’s literal formula $\lambda = 0.8$ pushes $\hat{V}$ *below* the mean (8.17 versus a mean of 10.33), whereas under the standard IQL-paper convention $\ell_2^\tau(Q - V)$ with $\tau = 0.8$ it pushes $\hat{V}$ *above* the mean (12.67). The question’s phrasing — “explain the intuition for why this is useful” — clearly intends the *above the mean* answer, because that is what makes IQL work.

Exam strategy. Write *one sentence* naming the convention you are using, then answer **above the mean**, then give the intuition. Optionally add one line noting that under the slide’s literal ordering the same behaviour corresponds to $\lambda < 0.5$. That demonstrates you understand the *mechanism* rather than the label — which is exactly what the marks are for.

The intuition itself. The upper expectile makes $\hat{V}(s)$ approximate $\max_a \hat{Q}(s, a)$ **restricted to actions present in the data**. So the submarine’s value for Trench reflects the *best* thing any recorded trajectory ever did from Trench (the τ_3 route to Target) rather than an average over the good route and τ_5 ’s bad one. That is what enables stitching — and it is achieved without ever querying $\hat{Q}$ at an action no submarine has actually tried.

6.3 (c) The “Diffusion Policy” slide describes flow matching

Linear interpolation $x_t = tx_1 + (1-t)x_0$, regression onto the velocity $x_1 - x_0$, and Euler integration are **rectified flow / flow matching**, not DDPM’s noise-prediction and reverse-SDE sampling. Answer using the slide’s procedure — that is what is being examined — but add half a sentence noting that

the mechanism is a learned *vector field* rather than iterative denoising. That half-sentence is the precision mark.

6.4 (d) Lecture 06 slides 11 and 13 share the same answer text

Slide 13’s trajectories are all *successes* (falls forwards, stumbles forwards, walks forwards, runs forwards), so the copied first sentence (“will encourage the policy to fall forward and not take a step forward”) does not apply. Slide 13’s actual point is the *second* sentence: **policy gradient is noisy, high-variance and sensitive to reward scale**, and with all-positive returns even the worst behaviour is reinforced. Use that.

6.5 (e) Lecture 09 slide 8 has a typo in the preference inequality

The slide prints $\sum_{(s,a) \in \tau_w} r_\theta > \sum_{(s,a) \in \tau_w} l_\theta$, with the same subscript τ_w on both sides and a stray symbol l_θ . The intended statement is

$$\sum_{(s,a) \in \tau_w} r_\theta(s, a) > \sum_{(s,a) \in \tau_l} r_\theta(s, a).$$

Write the corrected form.

6.6 (f) Scenario 5 Q2 notation

$\mathcal{N}(0; -30, 4)$ — the third argument is the **variance**, which is consistent with the stated $\sigma_1 = \sigma_2 = 2$. Since the question supplies both component densities at $a = 0$ as ≈ 0 , the answer to three decimals is **0.000**. The marks are for *writing the mixture density with the numbers substituted* and for stating that $a = 0$ falls in the trough between the modes — the same point as Q1, now quantified.

6.7 (g) A note on the practice paper’s Section 1 heading

Section 1 is titled “Policy Gradient Methods” but Q2.3 (PPO clipping) and Q2.4 (GAE / TD error) are Lecture 07 content. Expect cross-lecture placement throughout.

7 Worked Solutions to the Practice Paper

Marks are for *method*. Every numeric answer below shows the formula, the substitution line, and the result — reproduce that shape in the exam.

7.1 Scenario 1 — The Warehouse Robot

Q1 (3 marks). $\hat{Q}_t^i = \sum_{t'=t}^T r_{t'}$:

	$\hat{Q}$ at $t = 1$	$\hat{Q}$ at $t = 2$	Total return
τ_1	$1 + 10 = 11$	10	+11
τ_2	$-1 + 10 = 9$	10	+9
τ_3	$1 - 1 = 0$	-1	0
τ_4	$-1 - 1 = -2$	-1	-2

$$b = \frac{1}{8}(11 + 10 + 9 + 10 + 0 - 1 - 2 - 1) = \frac{36}{8} = 4.5.$$

Q2 (2 marks). τ_3 at $t = 1$: $0 - 4.5 = -4.5 < 0 \Rightarrow$ the gradient **decreases** $\log \pi_\theta$ for that action. τ_4 at $t = 2$: $-1 - 4.5 = -5.5 < 0 \Rightarrow$ the gradient **decreases** it as well. In both cases the action performed worse than the average timestep sample, so the update pushes probability away from it.

Q3 (2 marks). *Correct the premise.* τ_2 's reward-to-go at $t = 1$ is 9, and $9 - 4.5 = +4.5 > 0$, so the gradient **increases** the log-probability of that action — it does not reduce it. *Then make the real point.* That is itself the problem: the action at $t = 1$ moved the robot *away* from the shelf ($r_1 = -1$), but reward-to-go credits it with the +10 delivery that arrived at $t = 2$. A bad action is reinforced because a good outcome followed it. Reward-to-go with a scalar baseline cannot separate a bad action from a good consequence; a state-dependent advantage $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$ can, because it compares the action against what the policy would normally do *at that state*.

Q4 (2 marks). **Problem:** vanilla policy gradient is *on-policy*. After the first update θ has changed, so the four stored trajectories are samples from the wrong distribution and the gradient estimate is *biased* — not merely inefficient. **Fix:** off-policy policy gradient via *importance sampling*, reweighting each sample by $\prod_t \frac{\pi_{\theta'}(a_t | s_t)}{\pi_\theta(a_t | s_t)}$. **One condition under which the fix breaks down:** the trajectory-level ratio is a product of T terms, so its variance grows exponentially with the horizon — for long trajectories, or once $\pi_{\theta'}$ has moved far from π_θ (which is exactly what five successive gradient steps will do), the weights explode or collapse and the estimate becomes useless.

7.2 Scenario 2 — The Traffic Signal Controller

Q1 (3 marks). $\nabla_\theta J \approx \frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta(a_i | s_i)(R_i - b)$ with $N = 5$:

$$\begin{aligned} \sum_i(\cdot) &= 8[0.4, -0.2] + (-3)[-0.3, 0.5] + 5[0.1, 0.3] + (-1)[0.2, -0.4] + 2[-0.1, 0.2] \\ &= [3.2, -1.6] + [0.9, -1.5] + [0.5, 1.5] + [-0.2, 0.4] + [-0.2, 0.4] \\ &= [4.2, -0.8], \\ \nabla_\theta J &= \frac{1}{5}[4.2, -0.8] = [\mathbf{0.84}, \mathbf{-0.16}]. \end{aligned}$$

State your assumption if you omit the $\frac{1}{N}$: the slide formula includes it, so include it, but note that the direction is unchanged either way.

Q2 (1 mark). $\theta' = [1.0, 1.0] + 0.1[0.84, -0.16] = [\mathbf{1.084}, \mathbf{0.984}]$. *Reading:* θ_1 (north–south green time) increases and θ_2 decreases — the batch favoured giving more green to north–south.

Q3 (2 marks). $r_1 = 1.5$, $\hat{A}_1 = +8$, $\epsilon = 0.2$:

$$\text{unclipped} = 1.5 \times 8 = 12.0, \quad \text{clipped} = \text{clip}(1.5, 0.8, 1.2) \times 8 = 1.2 \times 8 = 9.6,$$

$$\min(12.0, 9.6) = \mathbf{9.6} \Rightarrow \text{the clipped term is used.}$$

Reading: the new policy has raised this action's probability by 50%, well beyond the 20% the clip allows, so the objective stops rewarding further movement and the update through this sample is throttled.

Q4 (2 marks). $\delta_t = r_t + \gamma \hat{V}(s_{t+1}) - \hat{V}(s_t) = 3 + 0.9(12) - 10 = 3 + 10.8 - 10 = \mathbf{3.8}$, and $\hat{A}_1^\pi(s_t, a_t) = \delta_t = \mathbf{3.8}$ — the one-step generalised advantage *is* the TD error; nothing further to compute.

7.3 Scenario 3 — The Medical Dosing Agent

Q1–Q3 (5 marks).

$$\delta_t = 6 + 0.95(24) - 20 = \mathbf{8.8}; \quad \hat{V}'(s_t) = 20 + 0.1(8.8) = \mathbf{20.88};$$

$$\theta' = \theta + \alpha_{\text{actor}} \delta_t \nabla_{\theta} \log \pi_{\theta} = [3.0, 1.5, 2.0] + 0.44[0.6, -0.4, 0.2] = [\mathbf{3.264}, \mathbf{1.324}, \mathbf{2.088}].$$

Note $\delta_t = 8.8$ is a *scalar* multiplying the gradient *vector*.

Q4 (2 marks). As $\delta \rightarrow 0$, $\pi_{\theta'}$ is pinned to π_{θ} : no meaningful update occurs, learning halts, maximally safe and maximally useless. As $\delta \rightarrow \infty$, the constraint is vacuous and you have unconstrained off-policy updates: the policy overfits the batch, the importance weights become unreliable, and the data no longer reflects the states the policy will visit. The practical algorithm that enforces a soft version by clipping importance weights instead is **PPO**.

Q5 (2 marks). (i) **The value target is wrong:** $\hat{V}$ fitted on the whole buffer estimates the value of “some kind of average” of all the past policies that contributed data, not of the current π_{θ} . (ii) **The likelihood term is wrong:** the actor update uses $\nabla_{\theta} \log \pi_{\theta}(a_i | s_i)$ where a_i came from an old policy — π_{θ} would probably not have chosen a_i at s_i , so the update pushes probability towards actions the current policy has no reason to consider.

7.4 Scenario 4 — The Robot Chef

Q1 (2 marks). $y_i = r_i + \gamma \hat{Q}(s'_i, \bar{a}'_i)$, $\gamma = 0.9$:

$$y_1 = 2 + 0.9(14) = \mathbf{14.6}, \quad y_2 = 5 + 0.9(9) = \mathbf{13.1}, \quad y_3 = 8 + 0.9(0) = \mathbf{8.0} \text{ (terminal).}$$

Q2 (2 marks). a_i in the buffer was chosen by an *old behaviour policy*, so using it would make $\hat{Q}$ an estimate of $Q^{\pi_{\beta}}$ — the value of that old policy — and the actor would be improving the wrong thing. Re-sampling $\bar{a}_i \sim \pi_{\theta}(\cdot | s_i)$ makes $\hat{Q}$ the value of the *current* policy, so the gradient $\nabla_{\theta} \log \pi_{\theta}(\bar{a}_i | s_i) \hat{Q}(s_i, \bar{a}_i)$ is valid for π_{θ} . **The problem this solves:** it removes the need for importance weights entirely, because the actions are drawn from the correct distribution by construction. (Worth one extra sentence: this same re-sampling is what queries $\hat{Q}$ at OOD actions — harmless online, fatal offline.)

Q3 (1 mark). PPO is a **proximal** method, not an off-policy one: its surrogate is only a valid local approximation to J while $\pi_{\theta'}$ is close to the policy that generated the data. Replay-buffer data comes from arbitrarily old policies, so the ratios lie far outside $[1 - \epsilon, 1 + \epsilon]$; clipping then *zeroes the gradient* rather than correcting the mismatch, and importance sampling cannot rescue it because those weights have unusable variance.

7.5 Scenario 5 — The Surgical Robot

Q1 (3 marks). The robot learns 0° — the conditional mean, because ℓ_2 regression is minimised by $\mathbb{E}[a \mid s]$. Draw the two Gaussian bumps at $\pm 30^\circ$, the dashed combined density with a trough at 0° , and mark $a = 0$ as “learned action / mean of the data” (see the sketch in Part III). **Why it is dangerous:** 0° is not a compromise between two valid approaches — it is a third action lying in the gap between the modes, which no surgeon ever performed. It cuts where *neither* approach cuts, into tissue both experts deliberately avoided.

Q2 (2 marks).

$$p(a \mid s) = 0.5\mathcal{N}(a; -30, 4) + 0.5\mathcal{N}(a; +30, 4), \quad p(0 \mid s) = 0.5(0) + 0.5(0) = \mathbf{0.000}.$$

The learned action sits at a point of essentially zero data density — the quantitative version of the same failure.

Q3 (2 marks, three sentences). Start from a sample of Gaussian noise $x_0 \sim \mathcal{N}(0, 1)$. Repeatedly take small Euler steps along the learned, state-conditioned velocity field, $x_{t+\delta} = x_t + v_\theta(x_t, t)\delta$, for $t = 0, \delta, \dots, 1 - \delta$. The endpoint x_1 is the action, and because different noise draws are transported to different places, different draws land on different modes — so the policy can output -30° on one call and $+30^\circ$ on another, never 0° . (Precision note: the slide’s procedure is flow matching, a learned vector field, rather than DDPM’s iterative denoising.)

Q4 (3 marks). **Compounding error / distribution shift.** Supervised learning assumes i.i.d. data, but the policy’s own predictions determine the next state. A small prediction error moves the robot to a state slightly off the surgeons’ distribution; there it has no training data, so its error is larger; that pushes it further off; errors grow with the horizon rather than staying bounded. **How DAgger prevents it:** it changes *where* the training states come from — roll out π_θ , query the expert surgeon for the correct action at the states the *learner* actually visited, and retrain — so the training distribution converges to the deployment distribution. **Update rule:**

$$D \leftarrow D \cup \{(s', a^*)\} \quad \text{where } s' \sim \pi_\theta, \quad a^* \sim \pi_{\text{expert}}(\cdot \mid s').$$

(Worth adding: DAgger needs an interactive expert throughout training, which is precisely what is unavailable when the demonstrations are recordings.)

7.6 Scenario 6 — The Autonomous Submarine

Q1 (3 marks). With $\eta = +5$: keep $\tau_1(+12)$, $\tau_3(+20)$, $\tau_4(+7)$; discard $\tau_2(-8)$ and $\tau_5(-3)$. The filtered state-action pairs are the transitions along Surface→Shallow→Target, Surface→Trench→Target, and Deep→Shallow→Target. **At Trench**, only τ_3 survives, so the robot learns exactly the one action τ_3 took there — head for Target. **The criticism:** that single sample fully determines the policy at Trench with no evidence it was the best available action, and τ_5 ’s Trench→Shallow→Fail evidence has been thrown away entirely rather than used to establish that the τ_3 action is better.

Q2 (3 marks). Value-propagation chain: (1) From τ_3 , TD learning backs the +20 at Target up into $Q(\text{Trench}, \text{correct action})$, so $V(\text{Trench})$ becomes high. (2) τ_2 contains the transition Deep→Trench, whose TD target is $r + \gamma V(\text{Trench})$ — the *learned* value of Trench, not the return observed in τ_2 . (3) Because $V(\text{Trench})$ is high (learned from a *different* trajectory), Deep→Trench receives a high value despite τ_2 ending in failure — the failure is attributed to what the submarine did *after* Trench, not to arriving there. (4) **This depends entirely on bootstrapping:** with Monte Carlo, the Deep→Trench transition would be credited with τ_2 ’s actual return of -8 and the agent would learn the *opposite* conclusion. So yes, it depends on TD versus MC, and only TD stitches.

Q3 (3 marks). *Why OOD actions are dangerous offline:* the critic target contains $\mathbb{E}_{a' \sim \pi_\theta(\cdot | s')} [\hat{Q}(s', a')]$ where a' need not appear in D ; $\hat{Q}$ is unconstrained and therefore arbitrary at such actions; the policy is *maximising* $\hat{Q}$ and so actively seeks the over-optimistic points; those inflated values propagate into the next TD target and compound; and **nothing corrects the error, because there is no ocean trial to refute it** — online the submarine would try the action and reality would disagree, but offline the loop is closed. *How IQL avoids it:* it never evaluates $\hat{Q}$ at an action outside D . The bootstrap target $\max_{a'} \hat{Q}(s', a')$ is replaced by $\hat{V}(s')$, and $\hat{V}$ is fit by **expectile regression over only the (s, a) pairs present in the dataset**, so the upper expectile approximates a maximum *restricted to the data’s action support*.

Q4 (2 marks). *State the convention in one sentence, then:* $\hat{V}(\text{Trench})$ is pushed **above the mean** of $\hat{Q}$ over the actions seen at Trench. (Using the standard IQL convention $\ell_2^\lambda(\hat{Q} - V)$, $\lambda = 0.8$ makes underestimation the expensive direction and drives $\hat{V}$ upward; under the slide’s literal ordering $\ell_2^\lambda(V - \hat{Q})$ the same behaviour corresponds to $\lambda < 0.5$.) *Why this is useful:* $\hat{V}(\text{Trench})$ then reflects the *best* outcome any recorded trajectory achieved from Trench — τ_3 ’s route to Target — rather than an average that τ_5 ’s failure would drag down. That preserved high value is exactly what lets the agent stitch Deep→Trench onto Trench→Target, and it is obtained without ever querying $\hat{Q}$ at an action no submarine has tried.

7.7 Scenario 7 — The Logistics Drone

Q1 (2 marks). $w(a) = \exp((\hat{Q}(s_D, a) - \hat{V}(s_D))/\alpha)$ with $\hat{V} = 11$, $\alpha = 1$:

$$w(\text{North}) = e^4 \approx \mathbf{54.60}, \quad w(\text{South}) = e^{-5} \approx \mathbf{0.0067}, \quad w(\text{East}) = e^{-1} \approx \mathbf{0.368}.$$

Q2 (2 marks). AWR overwhelmingly favours **Fly North** (normalised weight $\approx 99.3\%$ against 0.67% for East and 0.01% for South). This *is* consistent with the data frequency: North is also the most-observed action (80 of 95 samples). **Should the drone trust “Fly East”?** Its weight is not the largest here, so it does not drive the policy — but the reasoning matters: $\hat{Q}(\text{East}) = 10$ is estimated from only 5 samples, so it is a high-variance estimate, and AWR *exponentiates* the advantage, amplifying any error in it. AWR trusts $\hat{Q}$ unconditionally and has **no uncertainty penalty**; if East’s $\hat{Q}$ had come out at 16 through noise, AWR would have chased it. The correct posture is that a weight is only as trustworthy as the data support behind the $\hat{Q}$ that produced it.

Q3 (2 marks). **One advantage:** AWAC’s critic estimates $\hat{Q}^{\pi_\theta}$ for the *current* policy rather than $\hat{Q}^{\pi_\beta}$ for the weaker behaviour policy, and TD is lower-variance than the Monte Carlo returns AWR uses — so the advantage estimates are better and the drone can improve beyond the behaviour policy. **One disadvantage:** the TD target samples $a' \sim \pi_\theta(\cdot | s')$, so it queries $\hat{Q}$ at out-of-distribution actions, reopening the overestimation problem AWR structurally avoided — and with a fixed offline dataset there is no flight to disprove an inflated value.

7.8 Scenario 8 — The Content Moderation AI

Q1 (3 marks). (i) $J_{RM} = -\log \sigma(4.0 - 1.5) = -\log \sigma(2.5) = -\log(0.924) \approx \mathbf{0.079}$ — a low loss, meaning the model already ranks this pair correctly. (ii) $\binom{6}{2} = \mathbf{15}$ pairwise comparisons. (iii) $P(y_A \succ y_B) = \sigma(3.2 - 3.0) = \sigma(0.2) \approx \mathbf{0.550}$ — barely above chance, so the model sees these two as nearly equivalent.

Q2 (6 marks). (i) The KL term penalises *drift of the fine-tuned policy away from the pre-trained model’s distribution*, measured per response as a log-ratio. At $\beta = 0$ there is no anchor: the model maximises the reward model alone, exploits its errors, and produces degenerate high-scoring text with degraded fluency and general capability. At $\beta = 100$ the penalty dominates, $p_\theta^{RL} \approx p^{PT}$, and essentially no learning from the preferences occurs. (ii) The likely failure mode is **reward hacking**

/ **over-optimisation of the learned reward model.** Mechanism: RM_ϕ was trained on a finite sample of comparisons and is only reliable on that distribution; the RL policy is explicitly searching for the highest-scoring outputs, which drives it *off* that distribution and into the region where RM_ϕ is wrong. The reward-model score keeps climbing while genuine moderation quality falls. (This is the same structure as OOD exploitation in offline RL.) (iii) DPO loss:

$$\ell_{DPO} = -\log \sigma \left(\beta \log \frac{p_\theta^{RL}(y_w | x)}{p^{PT}(y_w | x)} - \beta \log \frac{p_\theta^{RL}(y_l | x)}{p^{PT}(y_l | x)} \right).$$

Key advantage in one sentence: it eliminates both the separate reward-model training stage and the RL loop, optimising the policy directly on preference pairs with a supervised-style classification loss — so there is no separate frozen reward model for the policy to hack, and training is cheaper and more stable.

Q3 (4 marks). (i) **Critique is easier than generation** — a model that cannot reliably produce the best response can still reliably judge which of two responses is less harmful. (ii) Two advantages: unlimited labelling throughput at negligible marginal cost, scaling instantly to internet-scale content volumes; and perfectly consistent application of a written constitution, with no annotator-to-annotator variance, no fatigue and no drift (and, relatedly, no human moderators exposed to harmful material). (iii) **One risk unique to AI annotation:** the annotator’s biases and blind spots are *systematic and correlated across every label*. Human errors are noisy and partially independent, so they average out across a pool of moderators; a single model’s blind spots do not. The reward model inherits them wholesale and the RL policy then optimises directly against them, with no independent ground truth in the loop to detect the drift.

7.9 Scenario 9 — The Chatbot Alignment Team

Q1 (3 marks). See the three-stage pipeline table in Part IV. The critical entry is stage 3’s data: *human pairwise preference comparisons* (x, y_w, y_l) , not (prompt, response) pairs.

Q2 (3 marks). **Two signs:** (i) the reward-model score keeps rising while human-judged quality falls — confident but factually wrong answers; (ii) the outputs look qualitatively unlike the pre-trained model’s, i.e. the measured KL from p^{PT} has grown large. **The diagnosis:** over-optimisation of the learned reward model (reward hacking). The reward model is unreliable off its training distribution, and the RL objective is actively searching for exactly those points. **Hyperparameter:** *increase* β , so the KL penalty keeps the policy inside the region where the reward model is still valid. (Also creditable: stop earlier, and retrain RM_ϕ on fresh on-policy comparisons.)

Q3 (3 marks). Substitute inside-out:

$$\beta(\text{ratio}_w - \text{ratio}_l) = 0.5(1.2 - 0.2) = 0.5; \quad \sigma(0.5) \approx 0.622; \quad \ell_{DPO} = -\log(0.622) \approx \mathbf{0.475}.$$

Q4 (3 marks). **What $Z(x)$ is:** the partition function for prompt x — the normalising constant $Z(x) = \sum_y p^{PT}(y | x) \exp(RM(x, y)/\beta)$, a sum over the entire (exponentially large) space of possible responses, and therefore intractable to compute. **Why it cancels:** it depends on x alone, not on y ; the DPO loss uses only the *difference* of the implicit rewards of two responses to the *same* prompt, so $\beta \log Z(x)$ is subtracted from itself. This is what makes training tractable without ever evaluating Z . **What this means practically:** you can compute *relative* rewards exactly — which of two responses is preferred — but the *absolute* reward of any single response y is known only up to the unknown constant $\beta \log Z(x)$. That is sufficient for preference optimisation and insufficient for anything requiring calibrated absolute scores.

7.10 Scenario 10 — The Robot Goal Classifier

Q1 (2 marks). It is called **exploiting the classifier’s weaknesses** — adversarial exploitation of a learned reward, i.e. reward hacking. Mechanism: the RL agent maximises the classifier’s *output*, not the actual goal condition. The classifier was trained on 50 positives and 50 negatives and has never seen an upside-down cup, so its output there is essentially arbitrary; the agent searches over states precisely to find where that arbitrary output is high, and 0.91 is what it found. The state *scores* like a goal without *being* one.

Q2 (3 marks). (i) All 500 visited states are added, so $|D^-| = 500$ (plus any earlier negatives), and $|D^+| = 50 + 30 = 80$. (ii) Because the batches are rebalanced **50/50**. A true goal state now appears in D^+ (original positives plus the 30 goals reached during RL) *and* in D^- (the rollout sweep). But D^+ is small (80) and D^- is large (500), so balancing to 50/50 **upweights each positive example** relative to each negative. For a state appearing on both sides the effective evidence is at least even, so the optimal classifier output is $p \geq 0.5$. Non-goal states appear *only* in D^- and are driven towards 0. The goal therefore remains the highest-reward state despite being nominally a negative.

Q3 (3 marks). See the comparison table at the end of Part IV. Add the closing sentence: the two failure modes are the same phenomenon — proxy exploitation — at different scales.

8 Compressed Formula Sheet

Quantity	Formula
<i>Lecture 06 — Policy gradients</i>	
Softmax policy	$\pi(a \mid s, \theta) = \frac{e^{h(s,a,\theta)}}{\sum_b e^{h(s,b,\theta)}}$
Objective	$J(\theta) = \mathbb{E}_{\tau \sim p_\theta} [\sum_t r(s_t, a_t)]$
Log-derivative trick	$p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) = \nabla_\theta p_\theta(\tau)$
REINFORCE gradient	$\nabla_\theta J \approx \frac{1}{N} \sum_i (\sum_t \nabla_\theta \log \pi_\theta(a_t^i \mid s_t^i)) (\sum_t r(s_t^i, a_t^i))$
Reward-to-go	$\hat{Q}_t^i = \sum_{t'=t}^T r(s_{t'}, a_{t'}^i) \quad (\text{inclusive of } t)$
Policy gradient with baseline	$\nabla_\theta J \approx \frac{1}{N} \sum_i \sum_t \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}) (\sum_{t'=t}^T r_{i,t'} - b)$
Constant baseline	$b = \frac{1}{N} \sum_i r(\tau_i)$
Baseline unbiasedness	$\mathbb{E}[\nabla_\theta \log p_\theta(\tau) b] = b \nabla_\theta \int p_\theta(\tau) d\tau = b \nabla_\theta 1 = 0$
Off-policy PG (per timestep)	$\nabla_{\theta'} J \approx \frac{1}{N} \sum_i \sum_t \frac{\pi_{\theta'}(a_{i,t} \mid s_{i,t})}{\pi_\theta(a_{i,t} \mid s_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(a_{i,t} \mid s_{i,t}) (\sum_{t'=t}^T r - b)$
KL trust region	$\mathbb{E}_{s \sim \pi_\theta} [D_{\text{KL}}(\pi_{\theta'}(\cdot \mid s) \parallel \pi_\theta(\cdot \mid s))] \leq \delta$
<i>Lecture 07 — Actor-critic</i>	
Value relations	$V^\pi(s) = \mathbb{E}_{a \sim \pi} [Q^\pi(s, a)]; \quad A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$
TD error = 1-step advantage	$\delta_t = r_t + \gamma \hat{V}(s_{t+1}) - \hat{V}(s_t) = \hat{A}_1^\pi(s_t, a_t)$
Critic update	$\hat{V}'(s_t) = \hat{V}(s_t) + \alpha_{\text{critic}} \delta_t$
Actor update	$\theta' = \theta + \alpha_{\text{actor}} \delta_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) \quad (\delta_t \text{ scalar})$
n-step advantage	$\hat{A}_n^\pi = \sum_{t'=t}^{t+n} \gamma^{t'-t} r(s_{t'}, a_{t'}) - \hat{V}_\phi^\pi(s_t) + \gamma^n \hat{V}_\phi^\pi(s_{t+n})$
GAE	$\hat{A}_{\text{GAE}}^\pi = \sum_{n \geq 1} w_n \hat{A}_n^\pi, \quad w_n \propto \lambda^{n-1}$
Surrogate objective	$\tilde{J}(\theta') \approx \sum_{t,i} \frac{\pi_{\theta'}(a_{i,t} \mid s_{i,t})}{\pi_\theta(a_{i,t} \mid s_{i,t})} \hat{A}^{\pi_\theta}(s_{t,i}, a_{t,i})$
PPO objective	$\sum_{t,i} \min(r_t \hat{A}, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A})$
Off-policy critic target	$y_i = r_i + \gamma \hat{Q}(s'_i, \bar{a}'_i), \quad \bar{a}'_i \sim \pi_\theta(\cdot \mid s'_i)$
Off-policy actor gradient	$\frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta(\bar{a}_i \mid s_i) \hat{Q}(s_i, \bar{a}_i), \quad \bar{a}_i \sim \pi_\theta(\cdot \mid s_i)$
<i>Lecture 08 — Imitation learning and offline RL</i>	
BC (deterministic)	$\min_\theta \frac{1}{ D } \sum_{(s,a) \in D} \ a - \pi_\theta(s)\ ^2$
BC (expressive)	$\min_\theta -\mathbb{E}_{(s,a) \sim D} [\log \pi_\theta(a \mid s)]$
GMM policy	$p(a \mid s) = \sum_{k=1}^K w_k(s) \mathcal{N}(a \mid \mu_k(s), \Sigma_k(s))$
Autoregressive policy	$\pi_\theta(a \mid s) = \prod_{d=1}^D \pi_\theta(a_d \mid s, a_{<d})$
Flow training	$x_t = tx_1 + (1-t)x_0$; minimise $\ v_\theta(x_t, t) - (x_1 - x_0)\ ^2$
Flow inference	$x_{t+\delta} = x_t + v_\theta(x_t, t)\delta$, from $x_0 \sim \mathcal{N}(0, 1)$, return x_1
Dagger	$D \leftarrow D \cup \{(s', a^*)\}, \quad s' \sim \pi_\theta, \quad a^* \sim \pi_{\text{expert}}(\cdot \mid s')$
Filtered BC	$\tilde{D} = \{\tau \mid r(\tau) > \eta\}$, then $\max_\theta \sum_{\tilde{D}} \log \pi_\theta(a \mid s)$
AWR value step	$\min_\phi \sum_{s_t \sim D} (\hat{V}_\phi^{\pi_\beta}(s_t) - \sum_{t'=t}^T r(s_{t'}, a_{t'}))^2$
AWR weight	$\exp(\frac{1}{\alpha} (\hat{Q}(s, a) - \hat{V}(s)))$
Expectile loss	$\ell_2^\lambda(x) = (1 - \lambda)x^2$ if $x < 0$, else λx^2
IQL V step	$\hat{V} \leftarrow \arg \min_V \mathbb{E}_D [\ell_2^\lambda(V(s) - \hat{Q}(s, a))]$
IQL Q step	$\hat{Q} \leftarrow \arg \min_Q \mathbb{E}_D [(Q(s, a) - (r + \gamma \hat{V}(s')))^2]$
IQL policy step	$\hat{\pi} \leftarrow \arg \max_\pi \mathbb{E}_D [\log \pi(a \mid s) \exp(\frac{1}{\alpha} (\hat{Q} - \hat{V}))]$
<i>Lecture 09 — Reward learning and RLHF</i>	
Bradley–Terry	$P(\tau_a \succ \tau_b) = \sigma(r_\theta(\tau_a) - r_\theta(\tau_b))$
Reward-model loss	$-\mathbb{E}_{(x, y_w, y_l)} [\log \sigma(RM_\phi(x, y_w) - RM_\phi(x, y_l))]$
Pairs from k items	$\binom{k}{2}$
RLHF objective	$\mathbb{E}_{\hat{y} \sim p_\theta^{RL}} [RM_\phi(x, \hat{y}) - \beta \log \frac{p_\theta^{RL}(\hat{y} \mid x)}{p^{PT}(\hat{y} \mid x)}]$
KL-regularised optimum	$p^*(\hat{y} \mid x) = \frac{1}{Z(x)} p^{PT}(\hat{y} \mid x) \exp(\frac{1}{\beta} RM(x, \hat{y}))$
DPO implicit reward	$RM_\theta(x, y) = \beta \log \frac{p_\theta^{RL}(y \mid x)}{p^{PT}(y \mid x)} + \beta \log Z(x)$

9 Symbol Table

Symbol	Meaning
s, a, r, s'	State, action, reward, next state
τ	A trajectory $(s_1, a_1, r_1, \dots, s_T)$; $r(\tau)$ is its total return
θ	Policy parameters; θ' the updated policy's parameters
ϕ	Parameters of a value function, critic, or reward model
$\pi_\theta(a s)$	The learned (parameterised) policy
π_β	The <i>behaviour</i> policy that generated an offline dataset; may be a mixture
π_{expert}	The demonstrator's policy in imitation learning
$h(s, a, \theta)$	Action preference; softmax logit
$p_\theta(\tau)$	Trajectory distribution induced by π_θ and the dynamics
$p(s_{t+1} s_t, a_t)$	Environment dynamics; <i>independent of θ</i> , which is why PG is model-free
$J(\theta)$	Expected return of π_θ — the objective being maximised
$\tilde{J}(\theta')$	The importance-weighted <i>surrogate</i> objective
γ	Discount factor
$V^\pi(s)$	Expected return from s following π
$Q^\pi(s, a)$	Expected return from s taking a once, then following π
$A^\pi(s, a)$	Advantage, $Q^\pi(s, a) - V^\pi(s)$
$\hat{V}, \hat{Q}, \hat{A}$	Learned / estimated versions of the above
$\hat{Q}_t^i$	Reward-to-go: $\sum_{t'=t}^T r(s_{t'}, a_{t'}^i)$, a one-sample estimate of Q^π
b	Baseline subtracted from the weight; the constant version estimates V^π
δ_t	TD error $r_t + \gamma \hat{V}(s_{t+1}) - \hat{V}(s_t)$; equals the one-step advantage. Scalar.
$\hat{A}_n^\pi$	n -step advantage estimate
λ (GAE)	Exponential weight decay across n -step estimates; the bias–variance dial
λ (expectile)	Asymmetry parameter of ℓ_2^λ ; 0.5 recovers the mean
ϵ	PPO clip half-width; ratios clipped to $[1 - \epsilon, 1 + \epsilon]$
δ (KL)	Trust-region radius on the expected KL divergence
$r_t(\theta')$	Importance ratio $\pi_{\theta'}(a_t s_t) / \pi_\theta(a_t s_t)$
α (RL)	Learning rate (α_{actor} , α_{critic})
α (AWR)	Temperature in $\exp(A/\alpha)$: small sharpens towards filtered BC, large flattens towards BC
$\bar{a}, \bar{a}'$	Actions <i>re-sampled</i> from the current policy, not read from the buffer
D	Dataset (demonstrations or offline transitions); $\tilde{D}$ its filtered subset
η	Return threshold in filtered behavioural cloning
$\mathcal{R}$	Replay buffer
w_k, μ_k, Σ_k	GMM component weight, mean and covariance (each a function of s)
$v_\theta(x_t, t)$	Learned velocity field in a flow / diffusion policy
$\mathcal{G}, C_\phi(s)$	Goal set; goal classifier used as a reward
D^+, D^-	Positive (goal) and negative (non-goal) state sets for the classifier
x, y	Prompt and response (LLM setting); y_w preferred, y_l dispreferred
$\tau_w \succ \tau_l$	τ_w is preferred to τ_l
$\sigma(\cdot)$	Logistic sigmoid
$RM_\phi(x, y)$	Learned reward model
p^{PT}, p_θ^{RL}	Pre-trained (frozen reference) model and the RL-fine-tuned policy
β	KL penalty coefficient in RLHF / DPO; the <i>price</i> of drifting from p^{PT}
$Z(x)$	Partition function for prompt x ; intractable, cancels in DPO
$\ell_2^\lambda(x)$	Expectile loss: $(1 - \lambda)x^2$ if $x < 0$, else λx^2
OOD	Out-of-distribution (an action with no support in the dataset)

10 Final Revision Checklist

Tick each item only if you can produce it *without notes*.

Derivations you may be asked to reproduce.

- ☐ The policy gradient from $J(\theta)$ to the sampled estimator, including the log-derivative trick and the one-sentence reason the dynamics vanish.
- ☐ The baseline unbiasedness chain ending in $b\nabla_{\theta}1 = 0$, and why the baseline may depend on s but never on a .
- ☐ $A^{\pi} \approx r + \gamma V^{\pi}(s_{t+1}) - V^{\pi}(s_t)$ from the definition of Q^{π} .
- ☐ The DPO chain: closed-form optimum $\rightarrow$ reward as a log-ratio $\rightarrow$ holds for arbitrary LMs $\rightarrow$ substitute into Bradley–Terry $\rightarrow Z(x)$ cancels.

Formulas you must be able to write exactly, including limits.

- ☐ Reward-to-go (sum from t , *inclusive*).
- ☐ δ_t (which $\hat{V}$ carries the γ , and which is subtracted).
- ☐ The n -step advantage (both the $-\hat{V}(s_t)$ and the $+\gamma^n \hat{V}(s_{t+n})$ terms).
- ☐ The PPO objective, both terms and the min.
- ☐ The off-policy TD target, including $\bar{a}' \sim \pi_{\theta}$.
- ☐ The AWR weight and the three IQL steps in order.
- ☐ The reward-model loss and the DPO loss (difference, not sum, inside σ).
- ☐ The DAgger update in one line, with both sources named.

One-sentence answers you should have ready.

- ☐ Why policy gradient is model-free.
- ☐ Why vanilla PG is invalid, not merely inefficient, on reused data.
- ☐ The one condition under which importance sampling breaks down.
- ☐ Why clipping *removes incentives* rather than enforcing a constraint.
- ☐ Why the min exists in PPO.
- ☐ Why PPO is unsuitable for a replay buffer.
- ☐ Why $\bar{a}$ is re-sampled from π_{θ} .
- ☐ The two problems with naive replay-buffer actor–critic.
- ☐ Why the ℓ_2 BC mean is dangerous (trough, not compromise).
- ☐ Why expressive *distributions*, not bigger networks, fix multimodality.
- ☐ The distinct mechanism of GMM vs autoregressive vs diffusion.
- ☐ Why compounding error is a violation of the i.i.d. assumption.
- ☐ What DAgger changes (the data distribution) and what it costs (an interactive expert).
- ☐ Why stitching needs TD and fails with MC.
- ☐ The five-step OOD chain, and why step 5 makes it an *offline* problem.
- ☐ Why IQL never queries an OOD action.
- ☐ What α controls in AWR, and what AWR does with a deterministic π_{β} .
- ☐ AWAC’s one advantage and one disadvantage.
- ☐ What the KL term penalises, and the behaviour at $\beta = 0$ and $\beta = 100$.
- ☐ The two signs of reward-model over-optimisation and the direction to move β .
- ☐ What $Z(x)$ is, why it cancels, and what that costs you practically.
- ☐ “Critique is easier than generation”, and the one risk unique to RLAIIF.
- ☐ The 50/50 balancing argument for goal classifiers.
- ☐ The single sentence tying OOD exploitation, reward hacking and classifier exploitation together.

Arithmetic you should be able to do without hesitating.

- ☐ Build a reward-to-go table and a mean baseline from a trajectory table.
- ☐ Multiply a scalar advantage into a gradient vector and add it to θ .
- ☐ Compute both PPO terms and pick the min.
- ☐ Compute TD targets with a terminal state.
- ☐ Compute $\exp(A/\alpha)$ weights and normalise them.
- ☐ Compute $-\log \sigma(\Delta)$ for a reward-model or DPO pair.
- ☐ $\binom{k}{2}$.

In the exam room. Compute before you narrate. Name a mechanism in every explanation. Answer the count that was asked. State an assumption where the paper is ambiguous, in one sentence, and then reason from the mechanism. And when a question in Section 4 is really the same question as one in Section 3, say so.